

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский
Нижегородский государственный университет им. Н.И. Лобачевского»
(ННГУ)

Институт информационных технологий, математики и механики

Кафедра алгебры, геометрии и дискретной математики

Направление подготовки: «Программная инженерия»
Профиль подготовки: «Технологии цифровой трансформации»

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

Тема:

**«Разработка некоторых серверных компонентов для системы
автоматизации услуг в сфере ЖКХ»**

Выполнил:
студент группы 3821М1ПР,
Федотов Владислав Игоревич

Научный руководитель:
профессор,
доктор ф.м.н. Золотых Н. Ю.

Консультант:
ст. преп., каф. МОСТ. Свистунов
А.Н.

Нижний Новгород

2023

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	4
Актуальность автоматизации процессов в сфере ЖКХ	6
1 Изучение предметной области и потребностей пользователей	7
1.1 Термины и сокращения	7
1.2 Идентификация пользователей	10
1.3 Получение информации об объектах недвижимости с подтверждением прав собственности	12
1.4 Передача показаний и поверка индивидуальных приборов учёта	13
1.5 Начисления	14
1.6 Оплата платёжных документов	15
1.7 Заявки в управляющую компанию и ГЖИ	16
2 Микросервисная архитектура высоконагруженных распределённых систем	18
2.1 Проблемы монолитной архитектуры	19
2.2 Понятие микросервисной архитектуры	20
3 Инструментальные средства разработки	22
3.1 Фреймворк Spring Boot	23
3.2 База данных PostgreSQL	25
3.3 Инструмент выполнения задач по расписанию Quartz Scheduler	26
4 Разработка программного модуля для функциональности оплаты платёжных документов	28
4.1 Постановка задачи	29
4.2 Дизайн моделей данных	31
4.2.1 Дизайн моделей данных, связанных с платежами	32
4.3 Функциональный дизайн API	38
4.4 Программная реализация	42
4.4.1 Высокоуровневая архитектура сервиса	42
4.4.2 Детали технической реализации требуемой функциональности	45
4.4.3 Обновление статусов заказов по расписанию и по событию	47

4.4.3.1 Реализация обработчика по расписанию	48
4.4.3.2 Реализация обновления статуса заказа по событию	49
4.4.3.3 Решение проблемы потерянного обновления (lost update)	49
4.4.3.4 Алгоритм процедуры синхронизации статуса со внешней платёжной системой	51
4.5 Обеспечение безопасности системы	54
4.5.1 Авторизация и аутентификация пользователей	54
4.5.2 Шифрование содержимого запросов	55
4.6 Мониторинг	58
4.7 Документирование и тестирование ПО	61
4.8 Описание результатов	63
5 Разработка программного модуля для функциональности поверки приборов учёта	64
5.1 Постановка задачи	65
5.2 Дизайн моделей данных	67
5.2.1 Дизайн моделей данных, связанных с заявками на поверку	68
5.3 Функциональный дизайн API	70
5.4 Программная реализация	73
5.4.1 Высокоуровневая архитектура сервиса	73
5.4.2 Детали технической реализации требуемой функциональности	75
5.5 Описание результатов	77
ЗАКЛЮЧЕНИЕ	78
СПИСОК ЛИТЕРАТУРЫ	79

ВВЕДЕНИЕ

В настоящее время жилищно-коммунальное хозяйство является одной из наиболее важных сфер государственной деятельности, поскольку оно напрямую связано с обеспечением комфортных жилищных условий для населения и поддержанием инфраструктуры городов и населённых пунктов. Однако, несмотря на значимость данной отрасли, её управление часто сопровождается рядом проблем, таких как недостаточная прозрачность и эффективность процессов, низкая степень автоматизации и цифровизации, а также ограниченный доступ к информации для широкой аудитории.

В связи с этим, оптимизация процессов управления жилищно-коммунальным хозяйством является актуальной задачей, которая требует разработки эффективных решений. Одним из таких решений может быть создание мобильного приложения для государственной информационной системы жилищно-коммунального хозяйства, которое позволит повысить эффективность управления и обеспечить более качественное обслуживание населения. В настоящее время мобильные приложения становятся все более популярными и востребованными среди пользователей. Мобильное приложение, которое объединяет значительную долю функций системы ГИС ЖКХ и позволяет пользователям быстро и удобно управлять своей недвижимостью, может значительно улучшить качество жизни людей.

Целью данной работы является решение прикладных задач по разработке и реализации backend части для мобильного приложения, предоставляющего обширную функциональность по решению жилищно-коммунальных вопросов. В рамках работы предполагается разработка программного кода модуля оплаты платёжных документов, базы данных для него, которая будет содержать информацию о платежах, а также организация системы мониторинга данных, которая позволит оперативно реагировать на изменения в работе системы и принимать соответствующие меры.

Для достижения поставленной цели необходимо решить следующие задачи:

- Изучение существующих подходов к управлению жилищно-коммунальным хозяйством и анализ предметной области в этой сфере.
- Разработка программного модуля и базы данных, которая будет содержать информацию о платежах пользователя.
- Разработка системы мониторинга, которая будет обеспечивать оперативный доступ к информации о работе инфраструктуры проекта и позволит принимать эффективные управленческие решения.

– Реализация серверной части мобильного приложения, которая будет обеспечивать взаимодействие с базой данных и предоставлять интерфейс для мобильного приложения.

Результаты данной работы могут быть использованы для оптимизации процессов управления жилищно-коммунальным хозяйством и улучшения качества жизни населения. Мобильное приложение государственной информационной системы жилищно-коммунального хозяйства позволит жителям городов и населённых пунктов получать оперативную информацию о работе системы, а также сообщать о проблемах, связанных с жилищно-коммунальным хозяйством. Это, в свою очередь, способствует повышению прозрачности и ответственности в управлении данной отраслью, а также улучшению качества жизни населения.

Актуальность автоматизации процессов в сфере ЖКХ

Жилищно-коммунальное хозяйство (ЖКХ) в России – это одна из самых сложных и многогранных сфер экономики. Она включает в себя множество процессов, связанных с управлением жилыми домами, обслуживанием коммунальной инфраструктуры и предоставлением услуг населению. Однако, несмотря на значительные инвестиции и усилия государства, эта сфера до сих пор остаётся одной из самых проблемных в России.

ГИС ЖКХ – государственная информационная система, которая была создана для упрощения управления жилищным фондом и коммунальной инфраструктурой. Она позволяет автоматизировать процессы учёта и контроля за работой жилищно-коммунальных служб, а также обеспечивает доступ к информации о состоянии объектов ЖКХ.

Однако, несмотря на все преимущества ГИС ЖКХ, её внедрение в России столкнулось с рядом проблем. В частности, это связано с недостаточной подготовкой специалистов, нехваткой квалифицированных кадров и сложностью процесса внедрения. Кроме того, многие регионы России не имеют доступа к этой системе из-за недостаточного уровня развития информационных технологий.

В связи с этим, автоматизация (цифровизация) в сфере ЖКХ становится всё более актуальной и важной. Внедрение новых технологий позволяет упростить управление жилищным фондом и коммунальной инфраструктурой, а также повысить эффективность работы служб ЖКХ.

Кроме того, электронные государственные услуги становятся всё более востребованными в России. Они позволяют гражданам получать необходимую информацию и услуги без необходимости посещения офисов и учреждений. Это экономит время и деньги, а также повышает уровень комфорта и удобства для населения.

Таким образом, автоматизация и развитие информационных технологий в сфере ЖКХ становится всё более актуальным и важным. Это позволит упростить управление жилищным фондом и коммунальной инфраструктурой, повысить эффективность работы служб ЖКХ, а также обеспечить доступность и качество коммунальных услуг для населения.

1 Изучение предметной области и потребностей пользователей

Жилищно-коммунальное хозяйство (ЖКХ) – это отрасль экономики, которая занимается управлением и содержанием жилых и нежилых зданий, а также обеспечением населения коммунальными услугами. В данном разделе будут рассмотрены основные понятия и задачи предметной области ЖКХ, которые будут решаться в рамках проекта. Данный раздел представляет собой теоретическую часть для ознакомления с предметной областью.

1.1 Термины и сокращения

Перед тем, как начать описывать технические аспекты проекта, необходимо понимать основные понятия и термины, которые используются в сфере ЖКХ. Это поможет лучше понять задачи и проблемы, которые решает проект, и правильно интерпретировать результаты его работы.

Один из основных терминов – объект недвижимости. Это может быть жилой или нежилой дом, квартира, комната, гараж или другое имущество, находящееся в собственности или аренде у государства или частных лиц.

Ещё одним важным понятием является пользователь. Это человек или организация, которые используют коммунальные услуги и имеют отношение к объекту недвижимости.

Для авторизации и получения электронных государственных услуг в сфере ЖКХ используется система ЕСИА (единая система идентификации и аутентификации). Она позволяет гражданам и организациям получать доступ к информации и услугам, не выходя из дома.

Платёжный документ – это документ, представляющий собой квитанцию за коммунальные услуги. Он может быть выписан управляющей организацией или поставщиком услуг.

Прибор учёта – это техническое устройство, которое используется для измерения объёма потреблённых ресурсов (электроэнергии, газа, воды и т.д.). Он позволяет точно определить объем потребляемых ресурсов и корректно рассчитать их стоимость.

Управляющая организация – это юридическое лицо, которое осуществляет управление многоквартирным домом или жилым комплексом. Она занимается

обслуживанием и ремонтом общего имущества, организацией работы коммунальных служб и взаимодействием с жильцами.

ГЖИ – это государственная жилищная инспекция. Она занимается контролем за соблюдением законодательства в сфере ЖКХ и рассмотрением жалоб граждан на деятельность управляющих организаций.

Заявка – это запрос на оказание услуг или решение проблемы, связанной с объектом недвижимости или коммунальными услугами. Она может быть подана жильцом, управляющей организацией или другим пользователем.

Изучение этих понятий поможет лучше понять задачи и проблемы, которые решает проект в сфере ЖКХ, и правильно интерпретировать результаты его работы. Более того, это поможет лучше понять потребности и ожидания пользователей, что является важным аспектом при разработке любого проекта в этой области.

Другие важные понятия, которые используются в сфере ЖКХ, включают в себя:

— Коммунальные услуги – это услуги, связанные с обеспечением жилых и нежилых помещений ресурсами (электроэнергия, газ, вода, отопление и т. д.), а также санитарной очисткой территории и уборкой снега.

— Тариф – это цена за коммунальные услуги, которая устанавливается государственными органами или организациями-поставщиками услуг.

— Лицевой счёт – это индивидуальный счёт, на котором отражаются начисления и оплаты за коммунальные услуги.

— Домовой счётчик – это прибор учёта, который устанавливается на отдельной квартире или комнате и позволяет точно определить объем потребляемых ресурсов для этой части объекта недвижимости.

— Расходомер – это прибор учёта, который используется для измерения объёма потребляемых ресурсов на общедомовые нужды (например, для подачи воды в дом).

— Услуги управления – это услуги, связанные с управлением объектом недвижимости, включая обслуживание и ремонт общего имущества, взаимодействие с жильцами и организацию работы коммунальных служб.

— Капитальный ремонт – это ремонт и обновление фундаментальных элементов здания (например, кровли, фасада, лифта и т.д.), который проводится с определённой периодичностью для поддержания качества и безопасности жилья.

— Жилищный фонд – это общее количество жилых помещений в определённом регионе или стране.

— Жилищные условия – это условия, в которых проживают люди, включая размер и качество жилых помещений, наличие коммуникаций и других условий, которые влияют на комфорт и безопасность проживания.

Понимание этих терминов поможет лучше понять сложности и вызовы, с которыми сталкиваются пользователи и организации в сфере ЖКХ, и определить наиболее эффективные решения для улучшения качества жизни в этой области.

1.2 Идентификация пользователей

В наше время государственные информационные системы становятся всё более распространёнными и важными для обеспечения безопасности и комфорта граждан. Однако, в связи с этим возникает проблема идентификации пользователей приложений, связанных с государственными информационными системами.

Существует несколько подходов к идентификации пользователей. Один из них – использование логина и пароля. Однако, такой подход имеет множество недостатков, включая возможность утечки личных данных и небезопасность хранения паролей.

Другой подход – использование биометрических данных, таких как отпечатки пальцев или сканирование лица. Этот метод более надёжный и безопасный, но требует наличия специального оборудования и может быть дорогостоящим.

Однако лучшим вариантом является использование ЕСИА – Единой Системы Идентификации и Аутентификации. ЕСИА – это единая система идентификации и аутентификации, которая позволяет пользователям получить доступ к различным государственным информационным системам. Технический процесс авторизации через ЕСИА включает несколько этапов.

Первый этап – регистрация в системе. Пользователь должен зарегистрироваться в ЕСИА, предоставив свои личные данные и подтвердив их с помощью электронной подписи или других средств идентификации.

Второй этап – аутентификация. Пользователь должен войти в систему, используя свою электронную подпись или одноразовый пароль, который отправляется на его мобильный телефон или электронную почту.

Третий этап – авторизация. После успешной аутентификации пользователь получает доступ к нужной ему государственной информационной системе.

ЕСИА имеет множество преимуществ в плане информационной безопасности. Во-первых, система использует современные методы шифрования данных, что обеспечивает высокую степень защиты информации от несанкционированного доступа. Во-вторых, ЕСИА позволяет пользователям контролировать свои личные данные и выбирать, какие данные они хотят предоставлять при доступе к различным государственным информационным системам. В-третьих, ЕСИА предоставляет возможность использования двухфакторной аутентификации, что повышает безопасность авторизации.

Таким образом, использование ЕСИА для идентификации пользователей приложений, связанных с государственными информационными системами, является наиболее оптимальным вариантом в плане информационной безопасности. Система обеспечивает

высокую степень защиты данных и удобство использования, что делает её популярной среди пользователей и государственных организаций.

Подводя итоги, можно сказать, что использование ЕСИА является наиболее оптимальным вариантом для идентификации пользователей приложений, связанных с государственными информационными системами. Она имеет множество преимуществ, включая высокую степень безопасности и защиты данных, возможность использования с любым устройством, доступность и удобство использования. Она обеспечивает высокую степень информационной безопасности и удобства использования, а также позволяет получить доступ к различным государственным информационным системам.

1.3 Получение информации об объектах недвижимости с подтверждением прав собственности

Современные технологии позволяют решать множество задач в сфере ЖКХ, но одной из главных проблем является получение и поддержание в актуальном состоянии информации о правах собственности на объекты недвижимости. Эта проблема связана с тем, что главным источником такой информации является Росреестр, который предоставляет данные в открытом доступе, но требует постоянной синхронизации.

Для того чтобы устранить эту проблему, сторонние приложения в сфере ЖКХ должны использовать специальные программные интерфейсы (API), которые позволяют автоматически получать информацию из Росреестра и синхронизировать её с базой данных приложения. Такой подход позволяет обеспечить актуальность данных и предоставлять пользователям достоверную информацию о правах собственности на объекты недвижимости.

Однако, необходимо учитывать, что Росреестр не всегда обладает полной информацией о правах собственности на объекты недвижимости, поэтому сторонние приложения должны иметь возможность вручную добавлять и изменять информацию в базе данных. Такой подход позволяет устранить возможные неточности в информации и обеспечить полную актуальность данных.

В целом, проблема получения и поддержания в актуальном состоянии информации о правах собственности на объекты недвижимости является одной из главных проблем в сфере ЖКХ. Однако, использование современных технологий и программных интерфейсов позволяет решить эту проблему и обеспечить пользователям сторонних приложений достоверную и актуальную информацию о правах собственности на объекты недвижимости.

1.4 Передача показаний и поверка индивидуальных приборов учёта

Индивидуальный прибор учёта – это счётчик, который устанавливается на объектах потребления (например, квартирах) и позволяет измерять количество потреблённых ресурсов (электроэнергии, воды, газа и т.д.) и точно определять их стоимость. Такой прибор позволяет снизить расходы на оплату коммунальных услуг, а также справедливо распределять затраты между жильцами многоквартирного дома.

Функциональность передачи показаний индивидуальных приборов учёта является важным элементом современных технологий управления коммунальными ресурсами. Она может быть реализована как часть мобильного приложения, которое позволяет пользователям вводить свои показания и отправлять их на сервер. Однако, реализация такой функциональности может представлять определённые сложности и нюансы.

В первую очередь, стоит отметить, что приборы учёта предназначены для измерения различных коммунальных ресурсов с различными единицами измерения, такими как киловатт-часы, кубические метры, килограммы и другие. Это требует разработки разных типов приборов учёта для каждого типа коммунального ресурса.

Кроме того, важно учитывать различные проблемы, которые могут возникать в процессе передачи показаний. Например, для обеспечения точности измерений, система должна проверять правильность и достоверность показаний, которые вводит пользователь. Это может включать в себя процедуры валидации, такие как ограничение вводимых значений и проверка на соответствие со значениями, введёнными ранее.

Также приборы учёта требуют регулярной поверки, что является важным элементом для поддержания их точности и работоспособности. Пользователь должен иметь возможность инициировать процесс поверки прибора учёта через приложение и получать информацию о её статусе и результате.

Важной функциональностью является определение количества дней, оставшихся до окончания периода передачи показаний. Это поможет пользователю сохранять контроль над процессом передачи показаний и вовремя реагировать на напоминания.

В целом, функциональность передачи показаний индивидуальных приборов учёта требует реализации множества элементов, которые должны быть взаимосвязаны и работать безупречно, чтобы обеспечить корректную передачу данных и учёт коммунальных ресурсов.

1.5 Начисления

Платёжный документ – это документ, который выставляется пользователю системы ЖКХ для оплаты коммунальных услуг. Коммунальные услуги могут включать в себя услуги по оказанию электричества, газа, воды, тепла, уборке территории, вывозу мусора и другие. Поставщиками услуг являются компании, которые оказывают коммунальные услуги на территории определённого региона. Получателем платежа является поставщик услуг.

Расчёт и начисление по потреблённым услугам происходит на основе данных, полученных от приборов учёта, которые установлены в каждой квартире или доме. Начисления могут производиться как по фактическому потреблению, так и по нормативам потребления. При этом необходимо учитывать различные факторы, влияющие на расход коммунальных ресурсов, такие как время года, количество проживающих в помещении людей и т. д.

Платёжный документ адресуется пользователю системы ГИС ЖКХ и содержит информацию о начислениях за определённый период времени, а также информацию о задолженностях и сроках оплаты. Для удобства пользователей необходимо предусмотреть функциональность просмотра и оплаты платёжных документов через мобильное приложение.

Реализация функциональности начислений в мобильном приложении для ГИС ЖКХ является важным элементом проекта. Пользователям важно иметь доступ к своим начислениям и задолженностям в режиме реального времени, чтобы иметь возможность своевременно оплачивать коммунальные услуги и избегать штрафов. Кроме того, реализация данной функциональности позволяет оптимизировать процесс начисления и учёта коммунальных ресурсов, что в свою очередь способствует более точному распределению затрат на коммунальные услуги между пользователями.

Важным элементом функциональности начислений является возможность просмотра задолженностей и оплаты платёжных документов через мобильное приложение. Это позволяет пользователям не только контролировать свои расходы, но и оплачивать коммунальные услуги в любое время и в любом месте, что удобно и экономит время.

В заключение, реализация функциональности начислений в мобильном приложении для ГИС ЖКХ является важным шагом на пути к оптимизации процесса учёта и начисления коммунальных ресурсов. Однако, для полного удобства пользователей, необходимо обеспечить пользователей не только функциональностью просмотра, но и полноценной оплатой платёжных документов через мобильное приложение.

1.6 Оплата платёжных документов

Оплата платёжных документов через мобильное приложение – это удобный и быстрый способ оплаты коммунальных услуг. Потребность и актуальность данной функциональности обуславливается тем, что она является логическим продолжением описанной выше функциональности начислений. Однако реализация функциональности оплаты может столкнуться с рядом проблем и сложностей.

Одной из таких проблем является объединение различных коммунальных услуг в одном платёжном документе, то есть один платёжный документ может относиться к нескольким получателям платежей с разными банковскими реквизитами. Это означает, что необходимо осуществлять перевод денег на разные банковские счета, что усложняет процесс осуществления перевода и требует интеграции с внешним сервисом платежей. При этом выбор такого сервиса должен быть осуществлён ответственно, учитывая различные условия сотрудничества и уровень стабильности и надёжности.

Ещё одной важной проблемой является безопасность проведения транзакции. Пользователю приложения приходится вводить свои платёжные данные, такие как номер карты, поэтому необходимо продумать механизм шифрования, чтобы защитить эти данные от злоумышленников.

Несмотря на эти сложности, реализация функциональности оплаты платёжных документов в мобильном приложении является важным шагом на пути к удобству и оптимизации процесса оплаты коммунальных услуг. Это позволяет пользователям быстро и удобно оплачивать свои счета в любое время и в любом месте, экономя время и сокращая возможность задолженностей.

Важно помнить, что реализация функциональности оплаты платёжных документов через мобильное приложение должна быть осуществлена с учётом всех возможных проблем и сложностей, чтобы обеспечить безопасность и удобство для пользователей.

1.7 Заявки в управляющую компанию и ГЖИ

Жилищно-коммунальное хозяйство – это обширный сектор, который включает в себя широкий спектр услуг для жителей многоквартирных домов. Но, к сожалению, не все потребители услуг ЖКХ имеют возможность законным образом контролировать качество этих услуг. Кроме того, несмотря на то, что многие поставщики услуг ЖКХ работают в соответствии с высокими стандартами качества, далеко не все жители получают услуги с уровнем качества, соответствующим цене этих услуг.

В результате из-за роста цен на коммунальные услуги многие потребители чувствуют недовольство и несправедливость. Они сталкиваются с проблемами в виде высоких счетов за коммунальные услуги, некачественно выполненных обязательных работ (например, уборка снега с крыш или поддержание чистоты в подъезде), плохого качества коммунальных ресурсов (например, ржавая вода или перепады напряжения) и некомпетентности сотрудников ЖКХ.

Ситуация усугубляется тем, что многие потребители ЖКХ не знают, куда обратиться в случае возникновения подобных проблем. Все чаще люди оставляют свои проблемы без внимания, так как они не знают, кто бы им мог помочь. Это приводит к тому, что проблемы не решаются вовремя, а жильцы продолжают тратить большие деньги на коммунальные услуги, оставаясь неудовлетворёнными их качеством.

Однако существуют компетентные организации и службы, которые помогают потребителям ЖКХ. Жильцы, столкнувшиеся с некачественными услугами, могут обратиться в органы местного самоуправления, которые обязаны брать жалобы на учёт и рассматривать в первоочередном порядке. Они также могут обратиться в организации, ответственные за управление коммунальной инфраструктурой (управляющие организации), которые также должны иметь свои службы по обработке жалоб и заявок.

Важно, чтобы потребители услуг ЖКХ имели возможность отправлять через мобильное приложение заявки в управляющую организацию с жалобами на проблемы, такие как отключения горячей воды, неисправность тепловых сетей, водопровода или канализации, плохая уборка мусора во дворе, ржавая вода. Также важно, чтобы пользователи могли легко получить информацию о статусе просмотра заявки и о сроках её исполнения.

Пользователь мобильного приложения должен иметь возможность отправлять фото и другие файлы, а также получать и просматривать ответ с решением заявки. Если пользователь не удовлетворён решением или имеет жалобу на работу управляющей организации, то он должен иметь возможность подать жалобу в орган государственной жилищной инспекции (ГЖИ). ГЖИ – это общественная организация, созданная для защиты

прав потребителей коммунальных услуг и контроля за работой коммунальных сервисов. В её компетенции входит контроль исполнения управляющими организациями своих обязательств по договору обслуживания многоквартирных домов. В случае выявления нарушений, ГЖИ может оштрафовать управляющую организацию. Таким образом, управляющие организации мотивированы качественно исполнять свои обязательства перед потребителями, то есть жителями МКД.

Подобная функциональность подачи заявок в мобильном приложении ЖКХ поможет улучшить жизнь потребителей услуг ЖКХ. Это даст пользователям возможность мгновенно обращаться к управляющей организации и просить помощи в случае возникновения проблем. Они смогут контролировать процесс исполнения заявки и видеть результаты работы управляющей организации.

Функциональность подачи заявок в управляющие организации и ГЖИ также поможет упростить процесс управления заявками и жалобами, сократив время и усилия, необходимые для выполнения работы. Она поможет повысить качество обслуживания и снизить количество жалоб на работу ЖКХ в целом. Кроме того, она поможет потребителям ЖКХ чувствовать себя более уверенно и контролировать свой бюджет благодаря более прозрачному процессу расчёта коммунальных платежей и решения проблем.

2 Микросервисная архитектура высоконагруженных распределённых систем

В данном разделе мы ознакомимся с некоторыми теоретическими сведениями, которые являются важной предпосылкой для практической части нашей работы и неразрывно связаны с концепцией микросервисной архитектуры.

Изучив преимущества микросервисной архитектуры перед традиционной монолитной, мы сможем понять, почему в нашем проекте применяется микросервисный архитектурный подход. Это знание станет основой для правильной оценки того, насколько успешно мы реализовали концепцию микросервисной архитектуры в нашем проекте.

Понимание проблем и сложностей, которые могут возникать в процессе разработки по принципу микросервисов, поможет нам грамотно выстраивать практические пути решения и принимать правильные решения в тех случаях, когда возникают сложности.

Изучение теоретических аспектов концепции микросервисной архитектуры является необходимым для успешной реализации данного подхода и важным шагом на пути к обеспечению высокой производительности и эффективности разрабатываемых проектов.

2.1 Проблемы монолитной архитектуры

Перед тем, как изучать микросервисную архитектуру, необходимо обозначить ключевые черты монолитного архитектурного стиля, который является противоположностью микросервисной архитектуры.

Монолитная архитектура – это классический подход к разработке программного обеспечения, который заключается в разработке всего приложения как одного целого с единой кодовой базой.

Это может привести к сложностям в масштабировании, расширении функционала, тестировании, отладке, поддержке, и в конечном итоге побудить к декомпозиции монолитного приложения на зависимые части. Единая кодовая база осложняет понимание роли и ответственности каждого компонента в системе, а также затрудняет разработку нового функционала, что может приводить к скрытым дефектам. Невозможность масштабировать отдельные компоненты приложения без масштабирования всего приложения может сильно ухудшить производительность. Кроме того, сбой в одном процессе может повлиять на всю систему. Монолиты требуют расширенного цикла развёртывания и обновления, который затрудняет внесение изменений в технологический стек приложения и ухудшает возможность непрерывной интеграции.

Осознание данных проблем является важным шагом перед переходом к микросервисной архитектуре, которая удачно решает большинство из них.

Микросервисная архитектура представляет собой модель разработки, которая гибко масштабируется в зависимости от изменяющихся потребностей, позволяет разрабатывать приложения с единственной обязанностью, интегрируется легко и ускоряет процесс развёртывания приложений, что в конечном итоге приводит к более эффективной и продуктивной работе.

2.2 Понятие микросервисной архитектуры

Микросервисная архитектура возникла в результате решения резко возросшего уровня сложности, связанного с разработкой монолитного приложения. Эта архитектура является модификацией подхода SOA, но отличается тем, что система разделена на совокупность множества автономных слабо связанных сервисов-модулей, которые взаимодействуют с помощью стандартизированных интерфейсов.

Каждый микросервис является элементарным, выполняющим одну чётко сформулированную задачу. Такой подход даёт большую гибкость системы, позволяет масштабировать и расширять приложения гораздо более эффективно, а также упрощает разработку и мониторинг.

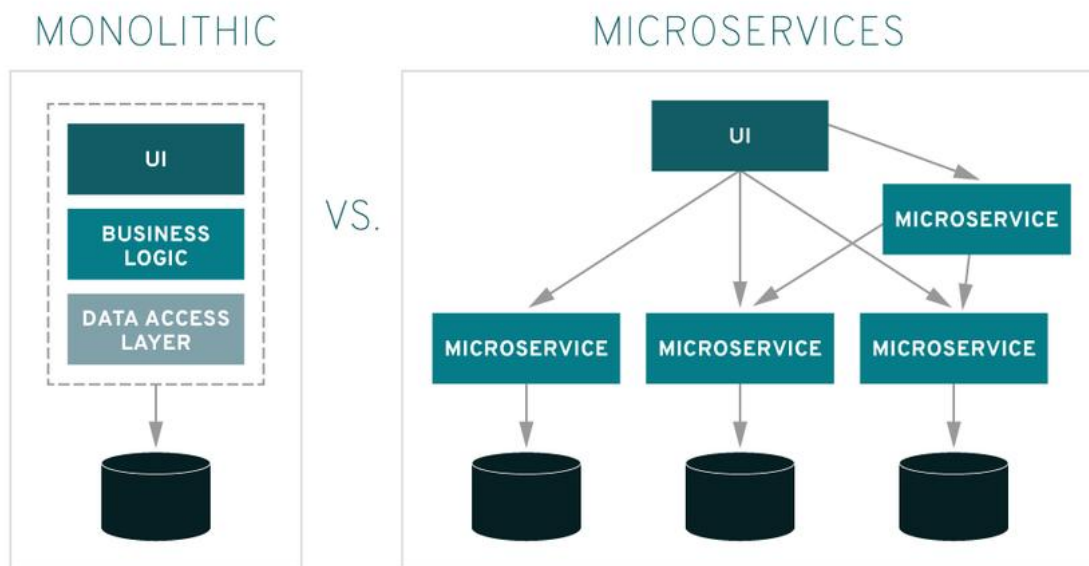


Рисунок 1. Сравнение монолитной и микросервисной архитектур

Схема иллюстрирует, что, в отличие от монолитной архитектуры, где все слои тесно связаны между собой, в микросервисной архитектуре клиент взаимодействует с совокупностью нескольких автономных слабосвязанных микросервисов, каждый из которых имеет свою собственную базу данных. Это даёт большую возможность для масштабирования, тестирования и отладки частей системы.

Таким образом, микросервисная архитектура является намного более гибкой и масштабируемой, чем монолитная архитектура, и может быть эффективным способом для достижения высокой производительности и эффективной разработки программного обеспечения.

2.3 Преимущества и недостатки микросервисной архитектуры

Микросервисная архитектура является прогрессивным подходом к разработке, который позволяет создавать более гибкие и масштабируемые системы. Она обладает рядом неоспоримых преимуществ, таких как:

- простота понимания (это снижает порог вхождения новых членов в команду разработчиков и упрощает процесс разработки и поддержки),
- лёгкость тестирования и отладки (надёжность и стабильность системы становятся выше),
- возможность раздельного масштабирования (когда нагрузка на отдельный модуль значительно превышает нагрузку на другие модули),
- быстрое введение в эксплуатацию (потому что можно обновить выборочные компоненты системы),
- каждый микросервис может иметь свой собственный стек технологий (исчезают проблемы несовместимости библиотек и зависимостей).

Однако, следует учитывать, что микросервисы не лишены недостатков. Распределённая архитектура вносит новые сложности и задачи, связанные с организацией совместной работы множества слабо связанных компонентов системы. Кроме того, необходимо обеспечить согласованность их работы и поддержание отказоустойчивости, балансировки нагрузки и стабильности системы в случае выхода из строя некоторых компонентов. Также актуальной становится проблема сетевых задержек.

В целом, микросервисная архитектура является эффективным инструментом для разработки сложных систем, но требует дополнительных усилий и знаний для её реализации и поддержки.

3 Инструментальные средства разработки

Перед рассмотрением прикладных задач, которые были решены в рамках работы, необходимо сначала описать выбранные инструментальные средства разработки. Это поможет читателю понять, какие технологии были использованы для решения задач и какие возможности были доступны для автора работы. Также это поможет другим специалистам в данной области оценить применимость выбранных инструментов и возможность их использования в своих проектах.

Разработка микросервисного приложения – это задача, требующая особого внимания к деталям и тщательного анализа требований. В данном случае разрабатываемые модули будут интегрированы в облачную инфраструктуру всего проекта, что требует от них не только высокой производительности и надёжности, но и хорошей масштабируемости.

Для достижения этих целей, одной из наилучших практик разработки микросервисов является использование специализированных фреймворков. Они позволяют абстрагироваться от базовых концепций веб-разработки и сосредоточиться на реализации содержательной части задачи (так называемой «бизнес-логики»). Такие фреймворки существуют для разных языков программирования, но в данном случае выбор пал на Spring Boot для языка Java.

3.1 Фреймворк Spring Boot

Перед тем, как рассматривать фреймворк Spring Boot, стоит упомянуть о его предшественнике – Spring Framework. Этот фреймворк был выпущен в 2002 году и с тех пор завоевал огромную популярность среди разработчиков Java-платформы, применяемой при создании сложных enterprise-приложений. Spring Framework предоставляет обширный функционал, который можно считать коллекцией фреймворков внутри фреймворка.

Однако в некоторых случаях использование Spring Framework становится избыточным и слишком сложным для функциональных требований веб-приложений. Именно в таких случаях возникает необходимость в инструменте, который позволил бы получить быстрый старт и не тратить много времени на низкоуровневую конфигурацию, но в то же время сохранять требуемую гибкость для решения прикладной задачи.

Spring Boot – это зрелый и распространённый на данный момент фреймворк, который позволяет ускорить процесс разработки и снизить количество написанного кода. Он предоставляет широкий спектр инструментов для работы с базами данных, веб-сервисами, безопасностью и многими другими аспектами, необходимыми для создания высококачественных микросервисов.

Выбор Spring Boot обусловлен не только устоявшимися практиками написания других модулей проекта и связанной с этим постановкой требований, но и предпочтениями разработчика. Данный фреймворк является популярным и общепотребительным инструментом в индустрии облачной микросервисной разработки, что говорит о его высокой эффективности и надёжности.

Spring Boot представляет собой инструмент, построенный по принципу "соглашения по конфигурации" (convention over configuration) и используемый для упрощения создания приложений на базе Spring Framework. Используя этот принцип, Spring Boot позволяет избавить разработчиков от необходимости конфигурирования тех аспектов Spring Framework, которые удовлетворяют базовым требованиям веб-приложения.

Кроме того, Spring Boot предоставляет широкие возможности для работы с базами данных, включая автоматическую конфигурацию и поддержку различных СУБД. Также он обеспечивает интеграцию с другими популярными инструментами, такими как Apache Kafka, RabbitMQ, Redis и многими другими.

Одним из ключевых преимуществ Spring Boot является его высокая производительность. Это достигается благодаря использованию многопоточности и асинхронного программирования, а также оптимизации работы с памятью и кэшированию данных. В результате приложения, построенные на базе Spring Boot, могут обрабатывать

большое количество запросов и обеспечивать высокую отзывчивость пользовательского интерфейса.

Также Spring Boot предоставляет мощный механизм для обработки ошибок и исключений, что позволяет создавать надёжные и безопасные приложения. Он также обеспечивает интеграцию с различными инструментами для тестирования и отладки приложений, что упрощает процесс разработки и повышает качество конечного продукта.

Таким образом, использование Spring Boot для разработки программных модулей в рамках микросервисной архитектуры является лучшим выбором, который позволит создать высококачественное и масштабируемое приложение с минимальными затратами времени и ресурсов. Выбирая Spring Boot в качестве каркаса разрабатываемого веб-приложения, можно сосредоточиться на решении прикладной задачи вместо длительного и ненадёжного конфигурирования модулей Spring. Благодаря предоставляемой "из коробки" функциональности, Spring Boot полностью соответствует поставленным задачам (в следующих разделах) и прекрасно вписывается в окружение других модулей проекта.

3.2 База данных PostgreSQL

Одной из важных задач при разработке программных модулей является обеспечение постоянного хранения данных. Информация о сущностях не должна быть потеряна после перезапуска приложения. Чтобы решить эту задачу, приложение должно быть интегрировано с базой данных.

В данном проекте было принято решение использовать PostgreSQL – одну из самых распространённых систем управления базами данных. PostgreSQL – это объектно-реляционная СУБД, которая позволяет обеспечивать постоянное хранение данных, а также обладает рядом других преимуществ. Она является одной из наиболее популярных СУБД на рынке благодаря своей надёжности, высокой производительности и богатому функционалу.

Одним из дополнительных преимуществ PostgreSQL является её свобода и открытость исходного кода. Благодаря этому база данных доступна и удобна для использования, особенно для крупных и сложных проектов. Система управления базами данных такого уровня требует высокой степени надёжности, и PostgreSQL как раз обеспечивает эту надёжность на высоком уровне.

Более того, PostgreSQL предоставляет большую гибкость в вопросе расширяемости. Она позволяет подключать и использовать различные модули, а также имеет встроенные средства для создания расширений на различных языках программирования. Благодаря этому PostgreSQL прекрасно справляется со сложными задачами.

Важным аспектом является то, что PostgreSQL поддерживает JSONB – двоичную (бинарную) разновидность формата JSON. В данном проекте это требование необходимо для хранения дополнительной информации о получателе платежа. Это позволяет нам хранить данные в удобном и компактном формате, а также производить быстрый поиск и анализ данных.

Итак, выбор PostgreSQL для решения поставленной задачи обусловлен объективными причинами. Она является бескомпромиссным решением благодаря своей бесплатности, широкой общественной поддержке и открытости исходного кода. PostgreSQL обладает множеством преимуществ, включая надёжность, высокую производительность и богатый функционал. Это делает её подходящим выбором для разработки модулей и соответствует общему стилю проекта.

3.3 Инструмент выполнения задач по расписанию Quartz Scheduler

Выполнение задач по расписанию – это важная функциональность, которая подразумевает автоматическое выполнение определённых программных методов (например, взаимодействие со внешней системой и базой данных) в соответствии с заданным расписанием. Для реализации этой функциональности мы можем использовать механизм Quartz Scheduler.

Quartz Scheduler – это популярный фреймворк для планирования задач в Java-приложениях. Он позволяет создавать и запускать задачи по расписанию, а также управлять ими во время выполнения. Quartz Scheduler имеет множество возможностей, таких как управление потоками, управление транзакциями и многое другое.

Кроме того, ключевой его особенностью является то, что, в отличие от, например, Spring Scheduler, Quartz Scheduler является распределённым и поддерживает кластеризацию, а также может восстанавливать выполнение прерванных задач. Таким образом, у Quartz Scheduler лучше отказоустойчивость и балансировка нагрузки.

Для реализации обработчика по расписанию мы можем создать класс, который будет реализовывать интерфейс Job из Quartz Scheduler. В этом классе мы определим логику обновления статусов заказов. Затем мы создадим объект JobDetail, который будет содержать информацию о задаче, которую мы хотим выполнить. Мы также создадим объект Trigger, который будет определять расписание выполнения задачи.

После того, как мы создали JobDetail и Trigger, мы можем создать объект Scheduler и зарегистрировать нашу задачу в нем. Затем мы запустим Scheduler, и он будет автоматически запускать нашу задачу в соответствии с заданным расписанием.

Расписание выполнения задачи задаётся с помощью cron выражения. Выражение cron состоит из шести полей, разделённых пробелами. Каждое поле может содержать либо конкретное значение, либо символы, обозначающие диапазон или периодичность. Синтаксис cron имеет следующий формат:

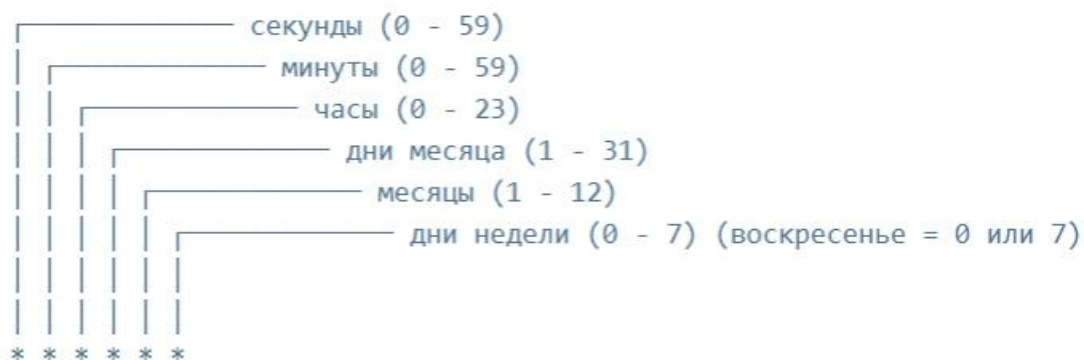


Рисунок 2. Синтаксис выражения cron

Например, выражение "0 0 * * * ?" означает, что задача должна выполняться каждый час, а выражение "0 * * * * ?" — каждую минуту.

Таким образом, использование Quartz Scheduler позволяет нам легко реализовать обработчик по расписанию. Мы можем задать любое расписание, которое соответствует нашим потребностям, и Quartz Scheduler будет автоматически запускать нашу задачу в соответствии с этим расписанием.

4 Разработка программного модуля для функциональности оплаты платёжных документов

В данном разделе мы рассмотрим процесс разработки серверной части программного модуля, который обеспечивает функциональность оплаты платёжных документов (см. раздел 1.6). Результатом этой работы будет готовый к интеграции микросервис, который можно будет использовать в составе системы разрабатываемого проекта.

Однако следует отметить, что разработка клиентской части, то есть мобильного приложения, не является частью данной работы. Эта задача была выполнена другими разработчиками на основе интерфейса, созданного в результате работы над серверной частью, описываемой в данном разделе.

Разработка микросервисной архитектуры позволяет создавать более гибкие и масштабируемые системы, но требует дополнительных усилий и знаний для её реализации и поддержки (см. раздел 2.3). В данном разделе будет представлен законченный микросервис, который готов к интеграции с другими программными модулями системы.

Необходимо обратить внимание, что из-за NDA (обязательства неразглашения конфиденциальной информации) невозможно предоставить полную информацию о технической реализации и модели данных. Эта информация будет представлена в упрощённом и изменённом виде, что позволит дать общее представление, при этом не нарушив обязательство о конфиденциальности. Это означает, что может быть упущено некоторое количество деталей, но общая структура и характеристики модели будут представлены и объяснены. Цель такой модификации данных – сохранить конфиденциальность и защитить права заключивших NDA сторон.

4.1 Постановка задачи

В рамках производственной необходимости проекта требуется разработать программный модуль (backend), который будет обеспечивать функциональность оплаты платёжных документов посредством интеграции с внешним платёжным сервисом.

Разрабатываемый программный модуль должен предоставить API для мобильного приложения по созданию заказа на оплату коммунальных услуг. Заказ может содержать несколько платежей, каждый из которых должен быть обработан внешним платёжным сервисом. После создания заказа мобильное приложение должно получить ссылку для перехода в вебвью внешнего платёжного сервиса для осуществления оплаты.

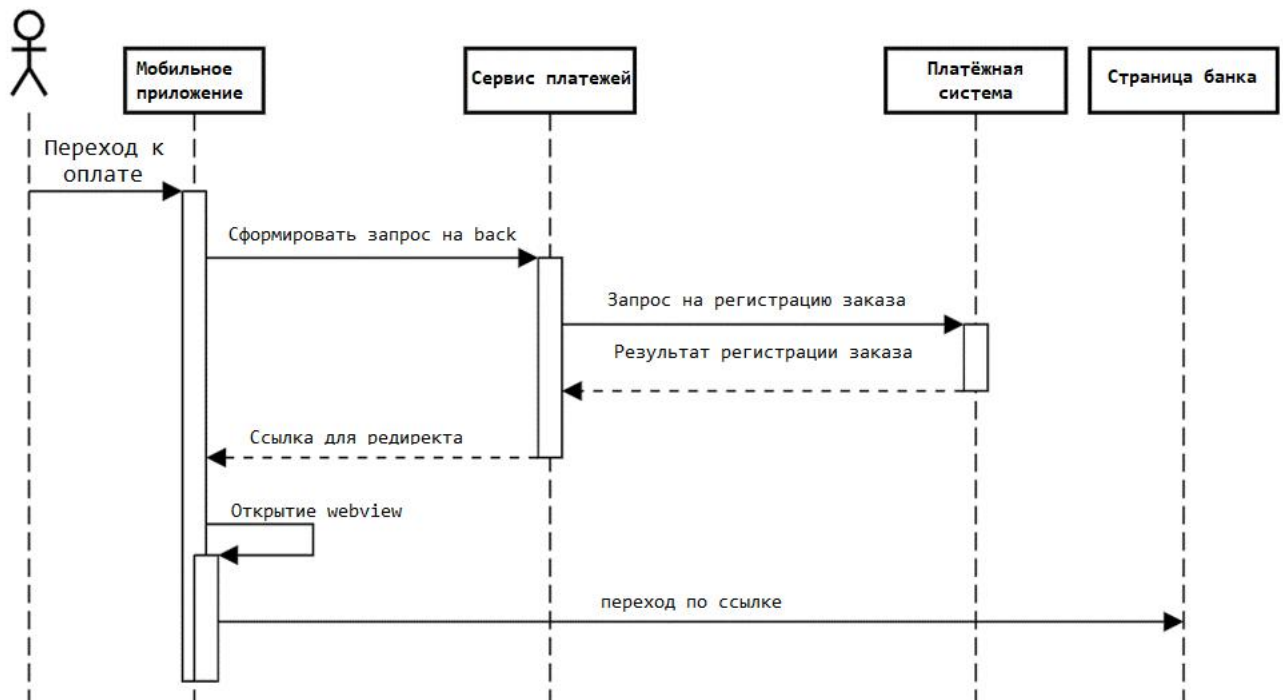


Рисунок 3. Диаграмма последовательности создания заказа на оплату

Для обеспечения синхронизации статусов заказов с внешней платёжной системой необходимо разработать механизм обновления статусов заказов по расписанию и по событию. Ниже представлена диаграмма последовательности обновления статуса по событию.

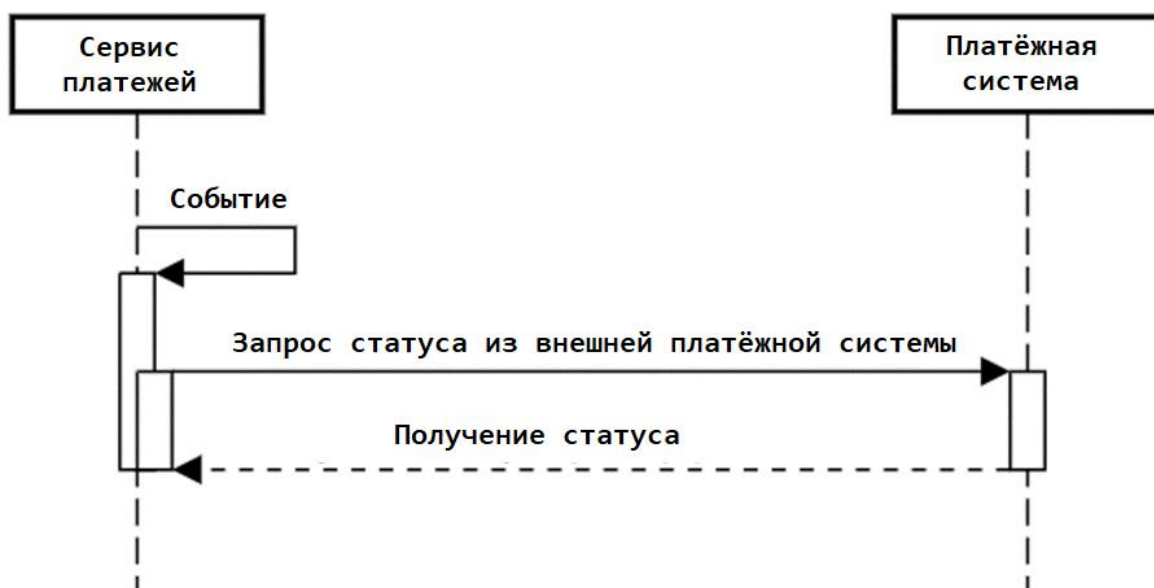


Рисунок 4. Диаграмма последовательности обновления статуса заказа по событию

Взаимодействие с внешней системой должно быть защищено шифрованием для обеспечения безопасности передачи данных.

Для обеспечения надёжности и отслеживаемости процесса оплаты необходимо осуществлять подробное логирование. Кроме того, информация о заказах и история изменений должны храниться в базе данных, что требует разработки соответствующей модели хранения данных.

Таким образом, задачей данного раздела является разработка серверной части программного модуля для обеспечения функциональности оплаты платёжных документов через внешний платёжный сервис. В рамках проекта необходимо разработать API для мобильного приложения, механизм обновления статусов заказов, защищённое взаимодействие с внешней системой, подробное логирование и модель хранения данных.

4.2 Дизайн моделей данных

Как известно, разработка любого компонента программного обеспечения является сложным процессом, в котором необходимо учитывать множество факторов. Одним из таких факторов является корректное фиксирование дизайна моделей данных, которое играет важную роль в создании готового к интеграции модуля.

Важность корректного фиксирования дизайна моделей данных заключается в том, что разрабатываемый компонент должен быть готов к интеграции в большую систему, разрабатываемую командой других программистов. При этом от него ожидается продуманный дизайн интерфейсов API и моделей данных, который гарантирует отсутствие нарушения «контрактов» с другими модулями, завязавшимися на функционал текущего.

В связи с этим, при разработке проекта было принято решение зафиксировать следующий дизайн моделей данных, задействованных в функционале разрабатываемого сервиса. При этом необходимо понимать, что обсуждение дизайна моделей данных представляет собой динамический процесс, который требует проведения нескольких итераций ввиду постепенного выявления требований. Поэтому в данном разделе представляется уместным документирование лишь заключительной версии дизайна, без протоколирования промежуточных набросков.

4.2.1 Дизайн моделей данных, связанных с платежами

В данном подразделе приводится дизайн типов данных, которые моделируют ресурсы, связанные с платежами. Для удобства данные структурированы в табличном виде.

Введём понятие *заказ* и *платёж*. Заказ представляет собой совокупность (корзину, пачку) платежей. Платёж – это сущность, описывающая операцию оплаты конкретной коммунальной услуги ответственному за неё поставщику услуг.

Таблица 1

Тип данных Order		
<i>Данный тип данных представляет собой сущность, описывающую совокупность (корзину, пачку) платежей.</i>		
Имя поля	Тип поля	Описание
id	Строковый	Уникальный идентификатор созданного заказа (автоматически генерируемый первичный ключ записи).
status	OrderStatus	Статус заказа.
status_message	Строковый	Сообщение статуса (полезно, например, в случае ошибки).
created	Дата и время	Дата и время, когда заказ был создан.
updated	Дата и время	Дата и время, когда заказ был последний раз изменён.
amount	Числовой	Сумма к оплате без учёта комиссии.
fee	Числовой	Сумма комиссии.
total_amount	Числовой	Общая сумма к оплате с учётом комиссии.
external_id	Строковый	Уникальный идентификатор заказа во внешней платёжной системе.
user_id	Строковый	Идентификатор (во внутренней системе) пользователя, создавшего заказ.
property_id	Строковый	Идентификатор (во внутренней системе) объекта недвижимости, по которому создан заказ.
bank_id	Строковый	Идентификатор банка, который обеспечивает проведение платежа.

bank_status	Строковый	Статус платежа в банке.
bank_status_message	Строковый	Сообщение статуса платежа в банке (полезно, например, в случае ошибки).
transaction_time	Дата и время	Время осуществления транзакции (проведения платежа) в банке.
payment_method	Строковый	Выбранный способ оплаты (например, карта или Система быстрых платежей).
payments	Список объектов типа Payment	Список платежей в рамках данного заказа.
update_retry_number	Целочисленный	Вспомогательный атрибут, выполняющий функцию счётчика попыток обновления информации о заказе из внешней платёжной системы (необходимо для ограничения количества запусков фоновой обработчика). Более подробно о назначении этого атрибута см. соответствующий раздел о фоновом обновлении заказа.
version	Целочисленный	Вспомогательный атрибут, представляющий собой версию записи. Введён для решения проблемы потерянного обновления. Более подробно о назначении этого атрибута см. соответствующий раздел о технической проблеме потерянного обновления (lost update).

Таблица 2.

Тип данных Payment		
<i>Этот тип данных представляет собой сущность, описывающую операцию оплаты конкретной коммунальной услуги ответственному за неё поставщику услуг.</i>		
Имя поля	Тип поля	Описание
id	Строковый	Уникальный идентификатор платежа (автоматически генерируемый первичный ключ записи).
order_id	Строковый	Идентификатор связанного заказа.
account	Строковый	Лицевой счёт.
payment_document_id	Строковый	Идентификатор платёжного документа.
payment_receiver_id	Строковый	Идентификатор получателя платежа.
amount	Числовой	Итоговая сумма к оплате (по получателю платежа).
additional_info	AdditionalInfo (JSONB)	Прочая информация по получателю платежа.

Таблица 3.

Тип данных AdditionalInfo		
<i>Этот тип данных служит для представления прочей информации по получателю платежа. В базе данных хранится как JSONB.</i>		
Имя поля	Тип поля	Описание
inn	Строковый	Идентификационный номер налогоплательщика (ИНН) получателя платежа.
kpp	Строковый	КПП получателя платежа.
name	Строковый	Наименование получателя платежа (организации).
account	Строковый	Расчётный счёт получателя платежа.
bank_name	Строковый	Название банка.
bank_bik	Строковый	БИК банка.

Таблица 4.

Тип данных OrderStatus		
<i>Перечислимый тип данных, представляющий собой статус заказа.</i>		
Значение	Описание	Возможные переходы по статусной модели
CREATED	Заказ создан, но ещё не зарегистрирован во внешней платёжной системе.	PROCESSING, ERROR
PROCESSING	Заказ зарегистрирован во внешней платёжной системе и находится в обработке (ожидает оплаты).	SUCCESS, DECLINED, CANCELLED
ERROR	Ошибка.	—
SUCCESS	Заказ успешно оплачен.	—
DECLINED	Заказ отклонён.	—
CANCELLED	Заказ отменён пользователем.	—

Таблица 5.

Тип данных Bank		
<i>Этот тип данных представляет собой описание сущности банка, с которым взаимодействует внешняя платёжная система (в силу ограниченности множества таких банков, соответствующая таблица в базе данных представляет собой справочник, предполагающий ручное заполнение).</i>		
Имя поля	Тип поля	Описание
id	Строковый	Идентификатор банка.
name	Строковый	Название банка.
address	Строковый	Адрес банка.
phone	Строковый	Номер телефона банка.

Таблица 6.

Тип данных OrderStatusHistory		
<i>Этот тип данных представляет собой сущность, описывающую переход заказа по статусной модели. Применяется для сохранения истории изменения статуса заказа.</i>		
Имя поля	Тип поля	Описание
id	Строковый	Идентификатор записи.
created	Строковый	Дата и время создания записи.
order_id	Строковый	Идентификатор связанного заказа, по которому произошло обновление. Внешний ключ.
previous_status	OrderStatus	Предыдущий статус заказа до осуществления обновления.
next_status	OrderStatus	Новый статус заказа после осуществления обновления.
message	Строковый	Сообщение при смене статуса.

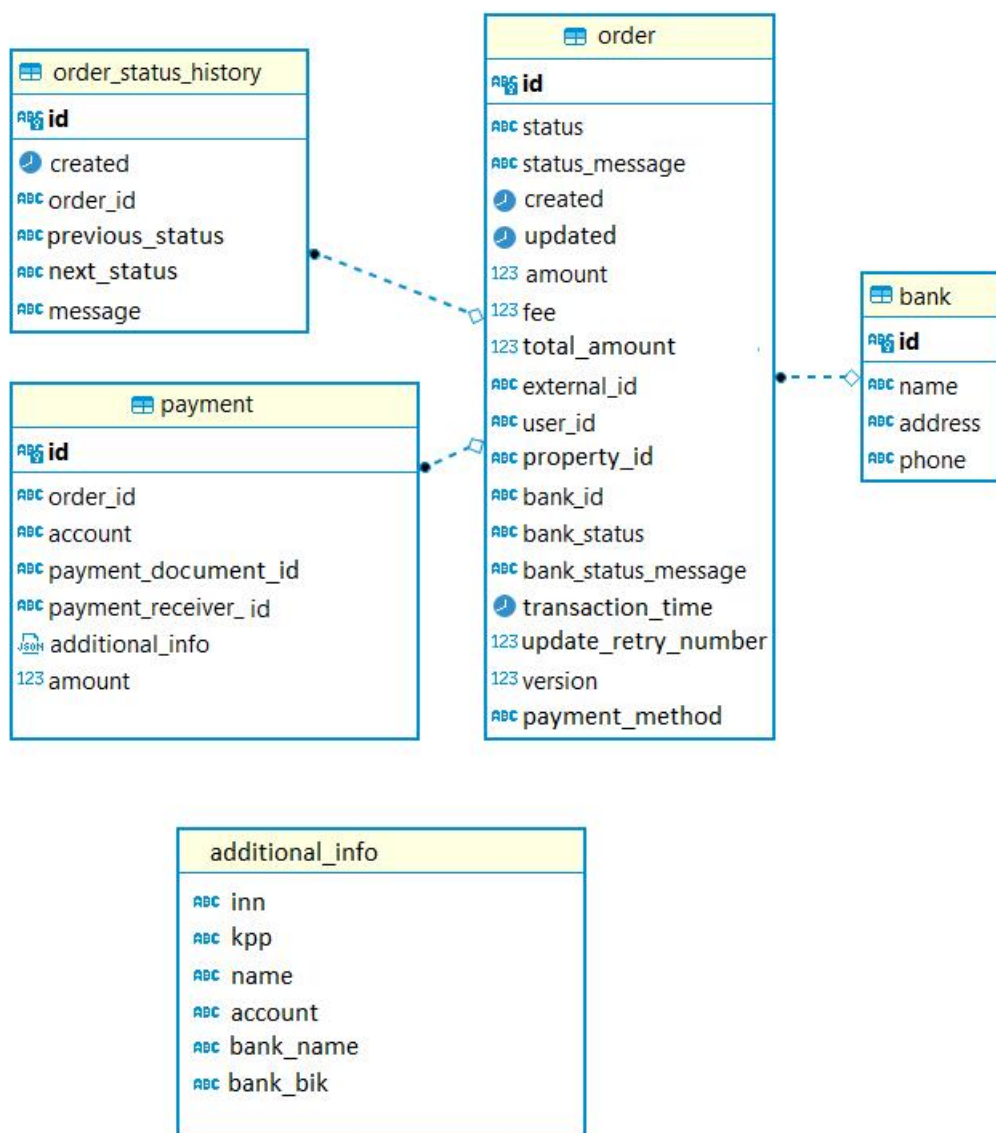


Рисунок 5. Диаграмма сущностей

4.3 Функциональный дизайн API

Рассматриваемая глава раскрывает результаты анализа функциональных требований, предъявляемых к серверной части создаваемого модуля. Важность документирования программного обеспечения в больших системах не может быть переоценена, поскольку время поддержки может значительно превышать время, затрачиваемое на разработку. Хорошо сформулированный функциональный дизайн модуля позволяет определить его «обязанности» и гарантировать, что он не повлияет на другие модули системы, что в целом повышает стабильность и устойчивость системы, а также облегчает последующую корректировку функционала.

В отношении создаваемого модуля сервиса оплаты наличие функционального дизайна API позволяет установить «контракт» между серверной и клиентской частями приложения. Это особенно актуально, поскольку разработка клиентской части (мобильного приложения) выполняется другими участниками проекта и выходит за рамки данной работы.

Таблица 7.

Название операции	HTTP метод	Путь	Входные данные	Выходные данные
Выполняет запрос от мобильного приложения на создание заказа на оплату во внешнюю платёжную систему	POST	/payments/order	В теле запроса передаётся структура данных, содержащая в себе все необходимые атрибуты для формирования сущностей Order и Payment, описанных выше.	В теле ответа передаётся ссылка для перехода в вебвью оплаты, на которую осуществляется редирект пользователя мобильного приложения.
Выполняет запрос на обновление статуса заказа	PATCH	/payments/order/{id}	Целочисленный идентификатор заказа (параметр пути).	OrderStatus (статус заказа после выполнения процедуры обновления).
Возвращает отсортированную и сгруппированную по месяцам историю заказов с фильтрацией по объекту недвижимости, платёжному документу и периоду	POST	/payments/transactions/history	В теле запроса передаются параметры фильтрации для поиска заказов: идентификаторы объекта недвижимости и платёжного документа, а также периоды начала и окончания данных.	Структура данных, представляющая собой список объектов, каждый из которых характеризует месяц оплаты (timestamp на первое число месяца) и содержит список транзакций (структура, возникающая в результате объединения преобразований из сущностей Order и

				Payment для представления на стороне мобильного приложения), осуществлённых за этот месяц.
Возвращает отсортированный список заказов с фильтрацией по объекту недвижимости, платёжному документу и периоду без группировки по месяцам (аналог предыдущего метода, но для других целей)	POST	/payments/transactions	В теле запроса передаются параметры фильтрации для поиска заказов: идентификаторы объекта недвижимости и платёжного документа, а также периоды начала и окончания данных.	Структура данных, представляющая собой список объектов, каждый из которых характеризует месяц оплаты (timestamp на первое число месяца) и содержит список заказов, осуществлённых за этот месяц.
Выполняет обработку редиректа от внешней платёжной системы в случае одобрения операции по оплате	POST	/payments/order/{id}/approve	Целочисленный идентификатор заказа (параметр пути).	OrderStatus (статус заказа после выполнения процедуры обновления).
Выполняет	POST	/payments/order/	Целочисленный	OrderStatus (статус

обработку редиректа от внешней платёжной системы в случае отклонения операции по оплате		{id}/decline	идентификатор заказа (параметр пути).	заказа после выполнения процедуры обновления).
Расшифровыв ает зашифрованн ый запрос/ответ при взаимодейств ии с платёжной системой (только для тестирования и отладки).	POST	/payments/test/de crypt	В теле запроса передаётся зашифрованное содержимое.	В теле ответа возвращается расшифрованное содержимое.

Для описания API выбран архитектурный стиль взаимодействия компонентов REST. Наиболее наглядным методом описания API является таблица, содержащая сводку по доступным операциям API с указанием HTTP протокола, по которому осуществляется запрос, пути относительно корневого адреса сервера, входных (параметры пути, параметры запроса, тело запроса) и выходных данных.

Одним из основных требований к создаваемому модулю является обеспечение безопасности. Для этого требуется, чтобы доступ к каждой из операций REST API был разрешён только для авторизованных пользователей. Пользователь предварительно выполняет запрос аутентификации, чтобы авторизоваться в системе. Авторизованным запросом считается тот, в заголовке (header) которого верно переданы закодированные аутентификационные данные.

4.4 Программная реализация

В данном разделе рассматривается непосредственная реализация компонента, функционал и дизайн которого детально описан в разделах 4.2 (модели данных) и 4.3 (функциональные требования).

В качестве языка программирования разрабатываемого приложения используется Java версии 17 (Open JDK). Фреймворк – Spring Boot версии 2.7.5.

4.4.1 Высокоуровневая архитектура сервиса

При разработке модуля с использованием фреймворка Spring Boot низкоуровневые детали реализации скрыты за рядом абстракций, что значительно сокращает объем исходного кода и делает его более лаконичным. Для этого применяется шаблон проектирования, который предусматривает разделение ответственности между различными компонентами приложения.

Один из ключевых элементов этого шаблона проектирования – это работа с данными, которая разделена на несколько составляющих. В частности, на уровень доступа к базе данных, работу с бизнес-логикой и предоставление доступа к своей функциональности для внешних сервисов.

Такая система разделения ответственности позволяет лучше структурировать код и создавать более надёжные и гибкие приложения. На уровне доступа к базе данных используются специальные классы, отвечающие за запросы к базе данных и получение информации. Бизнес-логика хранится в отдельном слое, который содержит все правила, связанные с функциональностью приложения и обработкой данных. И, наконец, для предоставления доступа к функциональности приложения со стороны внешних сервисов используются API-интерфейсы.

Для объяснения дизайна архитектуры, основанного на данном шаблоне проектирования, необходимо ввести упрощённые определения некоторых абстракций, подкреплённые примерами конкретных классов реализации:

- Контроллер представляет собой класс-компонент Spring, который обрабатывает внешние запросы и возвращает данные в качестве ответа. В данном контексте "PaymentsController" является классом, помеченным

аннотацией `@RestController`, который раскрывает REST API для взаимодействия с платежами извне.

- Сервис – это класс-компонент Spring, который реализует всю содержательную бизнес-логику работы с данными. В случае с "PaymentsService" он помечен аннотацией `@Service` и содержит имплементацию REST API, подготовку данных для взаимодействия со внешним платёжным сервисом и обработку его ответов для преобразования в формат взаимодействия с мобильным приложением. Это осуществляется в том числе с помощью маппера (см. ниже).
- Маппер – это компонент, который выполняет преобразование (mapping) данных между Java-объектами и объектами базы данных. Он позволяет работать с данными в более удобном для приложения формате, сокращает объем написанного кода и упрощает разработку приложений. В нашем случае класс `PaymentsMapper`, помеченный аннотацией `@Component`, является вспомогательным классом для преобразований запросов мобильного приложения в сущности БД (например, `Order` и `Payment`), а также сущностей БД в запросы во внешний платёжный сервис.
- Слой DAO – уровень доступа к данным, который является основным слоем, отвечающим за сохранение и получение информации из базы данных. В нашем случае класс `PaymentsDao`, аннотированный `@Service`, является промежуточным слоем между сервисом `PaymentsService` и четырьмя репозиториями. Он используется для того, чтобы пометить операции работы с сущностями разных репозиториях (таблиц) как транзакционные. Например, перед регистрацией заказа во внешней платёжной системе создание записи в таблицу с заказами и таблицу с платежами должно быть транзакционным: в случае неуспеха записи должны откатываться из обеих. Кроме того, именно на уровне `PaymentsDao` происходит формирование спецификации для динамической фильтрации при многокритериальном поиске сущностей (получение истории заказов).
- Репозиторий – класс-компонент Spring, абстрагирующий работу с базой данных и предоставляющий надёжный механизм, инкапсулирующий в себе поведение хранения, получения, создания и поиска данных. Применительно к решаемой задаче классы «`OrderRepository`», «`PaymentRepository`», «`OrderStatusHistoryRepository`» и «`BankRepository`» (аннотированные `@Repository`) избавляют от необходимости разработки явно прописанной логики взаимодействия с базой данных посредством написания SQL-кода.

Итак, с учётом введённых определений, высокоуровневую архитектуру подмодулей можно упрощенно изобразить следующим образом:

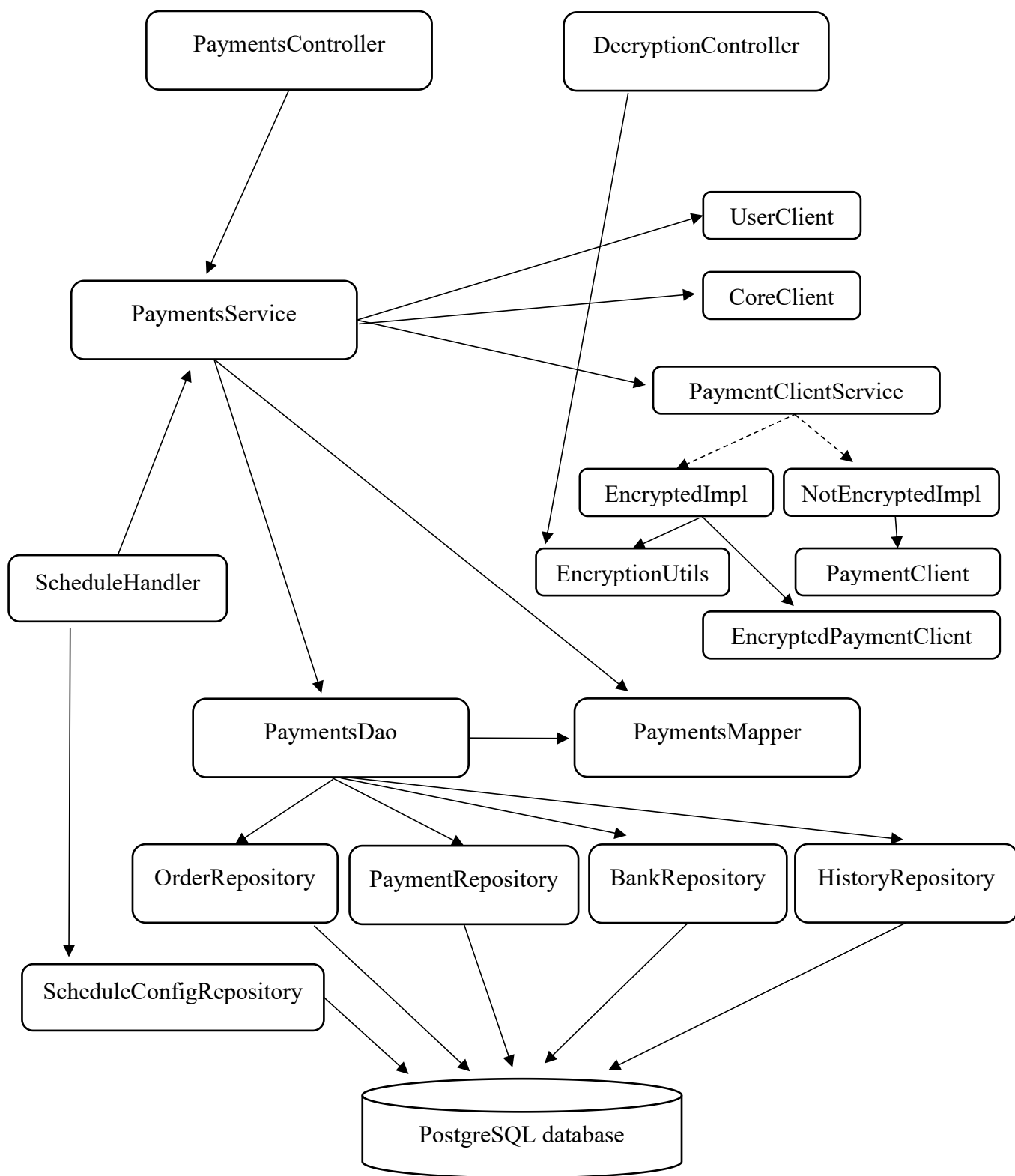


Рисунок 6. Высокоуровневая архитектура разрабатываемого сервиса оплат

Для достижения подобной лаконичности архитектуры используется ключевой паттерн фреймворка Spring – Dependency Injection («инъекция зависимостей»). На схеме компоненты, зависящие друг от друга, обозначены стрелками. Dependency Injection основан на принципе программной инженерии, называемом Inversion of Control («инверсия контроля»), который предполагает передачу контроля над объектами фреймворку вместо полного контроля над ходом выполнения программы, что позволяет упростить архитектуру.

Dependency Injection (DI) – это принцип программирования, который реализует Inversion of Control (IoC) путем связывания отдельных компонентов (бинов) в приложении Spring. Он основан на инверсии контроля над зависимостями между объектами. Вместо того, чтобы объекты создавали свои зависимости самостоятельно, они получают их из внешнего источника, контейнера Spring application context. Это позволяет управлять жизненным циклом объектов и легко изменять их зависимости без изменения кода самого объекта.

Например, если мы применяем аннотацию `@Service` к классу `PaymentsService`, то Spring Framework автоматически обнаруживает его и регистрирует в Spring Application Context в качестве компонента. Затем, если мы будем использовать constructor-based dependency injection в классе `PaymentsController` (то есть в качестве аргумента конструктора `PaymentsController` будет требоваться объект `PaymentsService`), мы можем ожидать, что контекст приложения автоматически разрешит эту зависимость, подставив зарегистрированный объект сервиса.

4.4.2 Детали технической реализации требуемой функциональности

Проведя первоначальный анализ высокоуровневой структуры компонентов нашего приложения, рассмотрим некоторые важные детали программной реализации ключевых функций, в частности REST API для работы с заказами на оплату.

Методы контроллера `PaymentsController` являются точками входа в программу из мобильного приложения и полностью соответствуют функциональному дизайну, описанному в разделе 4.3. Внутри тела этих методов происходит обращение к соответствующим методам в классе `PaymentsService`, который был внедрён в контроллер благодаря описанной выше Dependency Injection.

Среди функциональных задач, реализуемых в сервисном слое `PaymentsService`, который содержит непосредственную реализацию API, предоставляемого в классе `PaymentsController` (в частности, создание и регистрация заказов во внешней платёжной

системе, обновление статусов и получение истории из БД), а также маппере PaymentsMapper и DAO-слое PaymentsDao, можно выделить следующие:

- При формировании запроса на регистрацию платёжного заказа во внешнюю систему, происходит «обогащение» полученных из мобильного приложения атрибутов запросов дополнительными данными из других независимых модулей системы. А именно, из отдельного микросервиса, отвечающего за авторизацию пользователей, запрашиваются контактные данные инициировавшего процесс оплаты пользователя для передачи в платёжную систему. Кроме того, из другого микросервиса системы происходит получение детального адреса объекта недвижимости собственника, по которому начислен платёжный документ (в запросе от мобильного приложения приходит лишь идентификатор объекта).
- Формирование сущностей Order и Payment (на основании атрибутов запроса из мобильного приложения) и сохранение их в БД. Поскольку вторая сущность связана внешним ключом с первой, то это происходит в рамках SQL-транзакции (аннотация @Transactional) в DAO-слое PaymentsDao.
- Формирование запроса во внешнюю платёжную систему (на основании созданных выше сущностей Order и Payment и данных, полученных из других модулей системы: контактов пользователя и адреса объекта недвижимости), а также обработка результатов: в случае успеха статус заказа в БД меняется на успешный и возвращается ссылка для перехода в вебвью оплаты, на которую осуществляется редирект пользователя мобильного приложения; в противном случае статус заказа в БД меняется на ERROR, и мобильное приложение получает ответ с ошибкой.
- Приведение статусов заказов в актуальное состояние в соответствии с данными внешней платёжной системы. Более подробно про запуск процесса обновления статусов заказов по расписанию, а также про инициирование обновления статуса конкретного заказа по событию будет рассказано в соответствующем подразделе.
- Формирование спецификации (Specification) для динамической фильтрации при многокритериальном поиске сущностей в БД (используется для получения истории заказов). Возможность использования подобного механизма обеспечивается за счёт реализации Spring Criteria API путем создания объекта Specification и наследованием репозитория от класса JpaSpecificationExecutor.

- Формирование упорядоченной структуры истории заказов (то есть отсортированных и сгруппированных по месяцам) из списка объектов, характеризующих месяц оплаты (timestamp на первое число месяца) и содержащих список транзакций (структура, возникающая в результате объединяющих преобразований из сущностей Order и Payment для представления на стороне мобильного приложения), осуществлённых за этот месяц.

После обработки данных методы DAO-слоя (PaymentsDao) обращаются к соответствующим методам четырёх репозиторий для получения или сохранения ресурсов. Для создания репозитория, то есть "прослойки" между сервисом и базой данных, мы добавляем зависимость "spring-boot-starter-data-jpa" в наше приложение, которая добавляет Spring Data JPA в проект и, транзитивно, Hibernate – имплементацию JPA (Java Persistence API). Такое решение значительно сокращает объем кода, отвечающего за взаимодействие с базой данных, и автоматизирует большую часть типичных операций. Это также помогает избежать потенциальных ошибок, которые могут возникнуть, если бы приложение взаимодействовало с базой данных напрямую через JDBC (Java Database Connectivity).

4.4.3 Обновление статусов заказов по расписанию и по событию

Поскольку информация о созданных заказах на выполнение оплаты сохраняется в базу данных разрабатываемого модуля проекта, то, согласно требованиям, необходимо, чтобы статусы сохранённых заказов синхронизировались со внешней платёжной системой. Для этого нужно разработать механизм обновления статусов заказов, который бы работал как по расписанию (которое должно настраиваться с помощью конфигурации), так и по событию (редирект платёжной системы на специальные эндпоинты, сигнализирующие об успехе или ошибке процесса оплаты). Кроме того, должна быть возможность ручного инициирования процесса синхронизации статуса в целях разработки или отладки.

4.4.3.1 Реализация обработчика по расписанию

Когда мы реализуем обработчик обновления статусов заказов по расписанию, важно не только реализовать его функциональность, но и сделать его гибко конфигурируемым. Это позволит нам включать и выключать обработчик без перезапуска приложения, а также ввести ограничение на количество попыток обновления, чтобы заказ не обновлялся бесконечно (в случае превышения ограничения на количество безрезультатных попыток обновления статуса, предполагается принудительный перевод заказа в терминальный статус, согласно требованиям постановки задачи).

Для того, чтобы сделать обработчик гибко конфигурируемым, мы можем использовать конфигурацию на уровне базы данных. То есть мы можем создать таблицу, в которой будет определено, включён ли обработчик и какие установлены ограничения на количество попыток обновления заказа (также предусматривается возможность запуска обработчика без ограничения на количество повторных попыток обновления).

Кроме того, мы можем создать дополнительные параметры в файле конфигурации приложения и использовать аннотации Spring Framework, такие как `@ConfigurationProperties` и `@ConditionalOnProperty`, чтобы связать параметры конфигурации с нашим кодом. Это позволит нам легко изменять настройки приложения без необходимости изменять код.

Таким образом, редко изменяющиеся настройки, требующие перезапуска приложения, вынесены на уровень конфигурации свойств приложения (глобальное включение и выключение компонента, отвечающего за выполнение задачи по расписанию, и спон выражение, задающее расписание запуска обработчика), а настройки, требующие применения без перезапуска сервиса (на следующей итерации вызова обработчика) – на уровне конфигурации в таблице базы данных.

Наконец, мы можем использовать логирование, чтобы отслеживать работу обработчика и уведомлять администраторов о возможных проблемах. Например, мы можем записывать в логи параметры конфигурации, с которыми запустилось выполнение задачи обработчика, а также отслеживать количество попыток обновления заказа и уведомлять администраторов, если это количество превышает установленный лимит.

В целом, гибко конфигурируемый обработчик обновления заказов по расписанию позволит нам легко управлять его работой и предотвратить возможные проблемы в работе приложения.

Непосредственно сам алгоритм обновления статуса заказа будет рассмотрен ниже, в подразделе 4.4.3.4.

4.4.3.2 Реализация обновления статуса заказа по событию

Кроме обновления статуса заказов по расписанию, необходимо также обновлять статус по событию (callback). В случае успешной оплаты или отклонения операции, внешняя платёжная система должна осуществить редирект, в результате обработки которого на нашей стороне должна быть запущена процедура синхронизации статуса заказа с платёжной системой.

Для того, чтобы реализовать обновление статуса по событию, мы можем использовать механизмы обратного вызова (callback) в нашем приложении. Например, мы можем создать REST-контроллер, который будет принимать запросы от внешней платёжной системы и обновлять статус заказа в соответствии с полученными данными.

Кроме того, введём ещё один REST-контроллер, позволяющий в ручном режиме инициировать процедуру обновления статусов в целях тестирования и отладки.

Аналогично обработке по расписанию, будем использовать логирование, чтобы отслеживать работу и уведомлять администраторов о возможных проблемах. Например, мы можем записывать в логи результаты обработки запросов и уведомлять администраторов, если возникнут ошибки при обновлении статуса заказа.

В целом, обновление статуса заказов по событию позволит нам быстро реагировать на изменения заказов во внешней платёжной системе и обеспечить актуальность данных в нашем приложении.

Как уже было упомянуто выше, описание процесса обновления статуса заказа будет рассмотрено ниже, в соответствующем подразделе (4.4.3.4), поскольку процедура обновления статуса является общей как для обработки заказов по расписанию, так и для обработки по событию.

4.4.3.3 Решение проблемы потерянного обновления (lost update)

Проблема потерянного обновления (англ. *lost update*) возникает, когда два или более процесса пытаются обновить одну и ту же запись в базе данных одновременно. В этом случае может произойти потеря данных, так как одно из обновлений может перезаписать изменения, внесённые другим обновлением.

В контексте задачи обновления статусов заказов при одновременной обработке по расписанию и по событию, проблема потерянного обновления может возникнуть, если два процесса попытаются обновить статус заказа одновременно. Например, если процесс, запущенный по расписанию, обновляет статус заказа на "в обработке", а затем процесс, запущенный по событию, обновляет статус заказа на "завершён", то изменения, внесённые первым процессом, будут потеряны, и статус заказа будет установлен на "завершён", без учёта изменений, внесённых первым процессом.

Чтобы избежать проблемы потерянного обновления, можно использовать механизм блокировки записей в базе данных. Например, можно использовать оптимистическую блокировку, при которой каждая запись в базе данных имеет версию, которая увеличивается при каждом обновлении. Если два процесса попытаются обновить одну и ту же запись одновременно, то только один из них сможет выполнить обновление, а второй получит ошибку, что версия записи была изменена другим процессом.

Также можно использовать механизмы синхронизации и блокировки в коде приложения. Например, можно использовать ключевое слово `synchronized` в Java для блокировки доступа к общим ресурсам из нескольких потоков. Также можно использовать механизмы блокировки, предоставляемые фреймворком Spring, такие как `@Lock` и `@Transactional`.

Важно также учитывать, что при использовании блокировки записей или механизмов синхронизации может возникнуть проблема длительного ожидания, когда один процесс блокирует доступ к ресурсу, и другой процесс не может выполнить свою работу. Поэтому необходимо тщательно выбирать механизмы блокировки и синхронизации, чтобы избежать проблем с производительностью и отказоустойчивостью приложения.

Поскольку наш программный модуль представляет собой приложение Spring Boot, то для решения проблемы потерянного обновления будем использовать атрибут `version`, который позволяет реализовать оптимистическую блокировку.

Атрибут `version` представляет собой поле в сущности, которое увеличивается при каждом обновлении этой сущности. При обновлении сущности Spring проверяет, что значение атрибута `version` в базе данных соответствует значению, которое было получено

при загрузке сущности в приложение. Если значения не совпадают, то Spring генерирует исключение `OptimisticLockingFailureException`, которое можно обработать в коде приложения.

Для использования атрибута `version` в Spring необходимо добавить его в сущность с помощью аннотации `@Version`. Например:

```
@Entity

public class OrderEntity {

    ...

    @Version

    @Column(name = "version")

    private Integer version;

    ...

}
```

При обновлении сущности Spring автоматически увеличит значение атрибута `version` и проверит его соответствие значению в базе данных. Если значения не совпадают, то будет сгенерировано исключение `OptimisticLockingFailureException`.

Таким образом, использование атрибута `version` позволяет реализовать оптимистическую блокировку в Spring и избежать проблемы потерянного обновления при одновременном доступе к базе данных из нескольких процессов.

4.4.3.4 Алгоритм процедуры синхронизации статуса со внешней платёжной системой

Алгоритм процедуры синхронизации статуса со внешней платёжной системой является общим для обработки заказов как по расписанию, так и по событию. Это говорит о том, что алгоритм является общим методом, который может быть использован в разных контекстах и модулях приложения. Такая декомпозиция и модульная архитектура позволяют

улучшить качество и надёжность приложения, избежать дублирования кода и упростить поддержку приложения. Если бы алгоритм был реализован отдельно для каждого типа обработки заказов, то это привело бы к дублированию кода и усложнило бы поддержку приложения, а также могло бы привести к различным ошибкам и несоответствиям в логике приложения.

Рассмотрим подробнее процесс обновления статуса заказа:

- Сначала на основании идентификатора заказа, подлежащего обновлению, формируется запрос на обновление статуса во внешнюю платёжную систему.
- Далее, в зависимости от результата выполнения запроса,
 - Если получена ошибка HTTP-запроса (статус ответа не равен 200 OK), либо запрос успешен (статус 200 OK), но получена бизнес-ошибка (например, платёж по каким-либо причинам был отклонён на стороне банка, либо пользователь ввёл некорректные платёжные данные на странице оплаты, или любые другие ошибки, не связанные с техническими неполадками), то выполняется логирование ошибки и ответа внешней системы, а в ответ на вызов метода возвращается текущий статус из БД. Уровень логирования зависит от вида ошибки: если это бизнес-ошибка, то произойдёт запись в лог с уровнем WARN, а если техническая – то с уровнем ERROR.
 - Если запрос выполнен успешно и в результате отсутствуют сообщения о бизнес-ошибках, то на уровне DAO-слоя (PaymentsDao), в методе, помеченном аннотацией `@Transactional`, происходят следующие действия:
 - Осуществляется поиск в базе данных в таблице заказов записи с соответствующим идентификатором заказа. В случае отсутствия, бросается исключение, что заказ не найден.
 - Происходит обновление не только статуса заказа, но и некоторых других атрибутов, подлежащих обновлению (например, размер комиссии, способ оплаты: карта или СБП, время оплаты и т.д.), значения которых получены в ответе от внешнего платёжного сервиса.
 - В случае, если метод, реализующий этот алгоритм, был вызван из обработчика по расписанию (а не по событию), то происходит

увеличение счётчика (у каждого заказа свой) повторных попыток обновления:

- Если в результате запроса выяснилось, что объект не изменил статус (то есть пришедшее извне значение совпадает с записанным в базе данных), то происходит увеличение счётчика. Он будет увеличиваться до тех пор, пока не достигнет ограничения, сконфигурированного в настройках обработчика (см. ранее), после чего заказ перейдёт в терминальный статус (будет считаться отклонённым).
- Если заказ получил новый статус, то счётчик сбрасывается (потому что мы должны ограничивать именно безрезультатные попытки обновления).
- В случае, если в результате выполнения алгоритма заказ перешёл в новый статус, то должна осуществляться запись в историю переходов статусов заказов (отдельная таблица в базе данных: см. описание сущности OrderStatusHistory в разделе 4.2.1).
- Осуществляется запись в базу данных сущности заказа со всеми обновлёнными вместе со статусом атрибутами. Как было отмечено ранее, на этом этапе автоматически происходит увеличение значения version сущности, чтобы решить проблему потерянного обновления.

Итак, был рассмотрен процесс синхронизации статусов заказов со внешней платёжной системой, который вызывается как при обработке заказов по расписанию, так и по событию. Рассмотренный алгоритм учитывает проблему потерянного обновления, предотвращает заикливание выполнения и обеспечивает прозрачное логирование и протоколирование (запись истории в БД) операций.

4.5 Обеспечение безопасности системы

В современном мире обеспечение безопасности системы является одним из наиболее важных аспектов разработки программного обеспечения. В этом разделе мы рассмотрим два важных аспекта обеспечения безопасности системы: авторизацию и аутентификацию пользователей, а также шифрование содержимого запросов. Авторизация и аутентификация пользователей позволяют убедиться в том, что только правильно авторизованные пользователи имеют доступ к системе, а шифрование содержимого запросов обеспечивает защиту конфиденциальности передаваемых данных. В этом разделе мы рассмотрим основные принципы и методы реализации этих механизмов безопасности в приложениях.

4.5.1 Авторизация и аутентификация пользователей

Авторизация и аутентификация пользователей – это важные механизмы обеспечения безопасности системы, которые позволяют убедиться в том, что только правильно авторизованные пользователи имеют доступ к системе.

В силу NDA, подробные технические детали об обеспечении механизма авторизации и аутентификации не могут быть раскрыты. Однако можно отметить, что разработанный модуль интегрирован во всю систему с уже существующей системой авторизации, и все запросы от мобильного приложения поступают в него через защищённый фасад.

В целом, механизм авторизации и аутентификации пользователей должен быть надёжным и безопасным, чтобы предотвратить несанкционированный доступ к системе и защитить конфиденциальность пользовательских данных. Для обеспечения безопасности системы важно использовать надёжные механизмы авторизации и аутентификации пользователей. Один из таких механизмов – фреймворк Spring Security, который позволяет гибко настроить политику безопасности приложения при относительно небольшом объёме конфигурации.

Использование Spring Security в качестве основы для имплементации данного функционала обеспечивает высокую степень защиты системы от несанкционированного доступа и обеспечивает конфиденциальность пользовательских данных. Благодаря гибкой настройке политики безопасности, можно легко настроить механизм авторизации и аутентификации пользователей под конкретные требования системы.

4.5.2 Шифрование содержимого запросов

Шифрование содержимого запросов – это важный процесс, который позволяет защитить данные, передаваемые между клиентом и сервером. В настоящее время, когда всё больше людей используют интернет для покупок и оплаты услуг, шифрование данных становится необходимым условием для обеспечения безопасности и конфиденциальности информации.

Особенно важно шифровать данные, когда речь идёт о платежах. При передаче данных о банковских картах или других финансовых счетах, любая утечка информации может привести к серьёзным последствиям. Кроме того, мошенники могут использовать украденные данные для совершения мошеннических операций или кражи денег.

Шифрование содержимого запросов обеспечивает надёжную защиту данных во время их передачи. Это достигается путём использования специальных алгоритмов шифрования, которые переводят информацию в нечитаемый вид. Только получатель сможет расшифровать данные и получить доступ к ним.

Для шифрования содержимого запросов можно использовать различные алгоритмы шифрования, такие как AES, RSA и другие. В зависимости от требований системы и уровня защиты, можно выбрать наиболее подходящий режим алгоритма шифрования. Важно также убедиться в том, что механизм шифрования содержимого запросов интегрирован в систему с уже существующими механизмами безопасности, такими как упомянутые выше авторизация и аутентификация пользователей, чтобы обеспечить максимальную защиту системы от несанкционированного доступа.

В качестве такого алгоритма выбор пал на AES 256 (Advanced Encryption Standard) в режиме GCM (Galois/Counter Mode). Алгоритм шифрования AES 256 в режиме GCM – это один из самых безопасных и надёжных методов шифрования данных в интернете. Он используется для защиты конфиденциальной информации, передаваемой между клиентом и сервером.

Для начала необходимо понимать, что такое AES 256 и GCM. AES 256 – это алгоритм шифрования, который использует 256-битные ключи для защиты данных. Он считается одним из наиболее надёжных методов шифрования в настоящее время. GCM (Galois/Counter Mode) – это режим работы алгоритма, который обеспечивает аутентификацию данных и защиту от подмены.

В режиме GCM данные разбиваются на блоки, каждый блок шифруется с использованием AES 256 в режиме счётчика. Затем, для обеспечения целостности и подлинности, к каждому блоку добавляется аутентификационный тег, который вычисляется с помощью полиномиальной функции Галуа. Тег используется для проверки целостности и подлинности данных при их передаче или хранении.

Выбор режима GCM среди других режимов обусловлен его высокой производительностью и безопасностью. Он обеспечивает аутентификацию и шифрование данных в одном проходе, что позволяет уменьшить нагрузку на систему. Кроме того, он обеспечивает защиту от атак на подмену данных.

Реализация алгоритма, помимо ключа шифрования, использует вектор инициализации (IV, initialization vector). IV – это случайно сгенерированная последовательность байтов, которая добавляется к сообщению перед шифрованием. Она позволяет увеличить уровень безопасности шифрования и предотвратить повторное использование ключей. Длина вектора инициализации в данном случае составляет 12 байт.

Ключ шифрования не передаётся с запросом, а хранится на стороне получателя и отправителя. Это означает, что только авторизованные пользователи смогут получить доступ к зашифрованным данным. Ключ шифрования используется для шифрования и расшифровки данных.

Теперь рассмотрим процесс шифрования тела HTTP запроса:

- Сначала генерируется случайный вектор инициализации длиной 12 байт.
- Затем сообщение шифруется с использованием ключа шифрования AES 256 в режиме GCM и уникального вектора инициализации, сгенерированного на шаге выше. При этом происходит не только шифрование данных, но и вычисление кода аутентификации (MAC), который используется для проверки целостности данных.
- После выполнения шифрования в начало сообщения (первые 12 байт) добавляется сгенерированный ранее вектор инициализации.
- Запрос с зашифрованным содержимым и размещённым в начале уникальным вектором инициализации отправляется внешней платёжной системе.
- Платёжная система, получив запрос в зашифрованном виде, извлекает первые 12 байт содержимого, которые представляют собой вектор инициализации, а затем использует его в совокупности с известным ей (и отправителю) ключом шифрования для расшифровки содержимого запроса.

- При расшифровке данных также проверяется тег аутентификации, чтобы убедиться в том, что данные не были изменены в процессе передачи. Если MAC совпадает, то данные считаются подлинными и целостными.

Итак, алгоритм шифрования AES 256 в режиме GCM с длиной вектора инициализации 12 байт – это очень безопасный и надёжный метод защиты данных в интернете. Он обеспечивает конфиденциальность и целостность данных, а также защиту от подмены. Ключ шифрования хранится на стороне отправителя и получателя, что делает его недоступным для третьих лиц.

4.6 Мониторинг

В рамках разработки проекта по автоматизации услуг жилищно-коммунального хозяйства заказчиком было принято решение о необходимости реализации мониторинга, который позволил бы отслеживать текущую нагрузку по памяти и входящим запросам для того, чтобы можно было оперативно реагировать и предотвращать сбои.

Мониторинг является важной частью процесса разработки, так как он позволяет контролировать работу системы (показатели производительности, технического состояния, доступности), в том числе благодаря возможности графического анализа показателей, и предотвращать возможные проблемы посредством уведомления ответственного персонала при достижении критических показателей.

Технологический стек для мониторинга включает в себя Spring Boot Actuator, Micrometer, Prometheus и Grafana. Для реализации мониторинга необходимо настроить Spring Boot Actuator, чтобы он предоставлял (с помощью фасада Micrometer) данные о состоянии приложения, подключить Prometheus для сбора и хранения этих данных, а затем создать дашборды в Grafana для визуализации этих данных.

Spring Boot Actuator – это модуль фреймворка Spring Boot, который предоставляет набор эндпоинтов для мониторинга приложения. Он позволяет получать информацию о состоянии приложения, такую как количество запросов, время ответа, использование памяти и другие параметры.

Spring Boot Actuator работает следующим образом: при запуске приложения он автоматически создаёт набор эндпоинтов, которые можно использовать для получения информации о состоянии приложения. Эти эндпоинты доступны по адресу /actuator в приложении.

Для сбора и хранения данных, предоставляемых Spring Boot Actuator (через фасад Micrometer), будем использовать Prometheus. Prometheus – это система мониторинга, которая позволяет собирать метрики с различных источников и хранить их в своей базе данных. С помощью Spring Boot Actuator и Prometheus можно собирать метрики о состоянии приложения и использования ресурсов. Для работы с Prometheus необходимо настроить Spring Boot Actuator для экспорта метрик в формате Prometheus. Это можно сделать, добавив зависимость на библиотеку Micrometer и настроив её для экспорта метрик в формате Prometheus.

Затем необходимо настроить Prometheus для сбора метрик из приложения. После настройки Prometheus можно использовать его для мониторинга приложения. Он будет собирать метрики из приложения и хранить их в своей базе данных.

Для визуализации данных из Prometheus можно использовать Grafana. Grafana – это платформа для визуализации данных, которая позволяет создавать красивые и информативные доски (*dashboards*).

В рамках проекта были кастомизированы и настроены доски в Grafana, чтобы отображать информацию о текущей нагрузке по памяти и входящим запросам. Были созданы диаграммы, которые показывают количество запросов, время ответа и использование памяти.

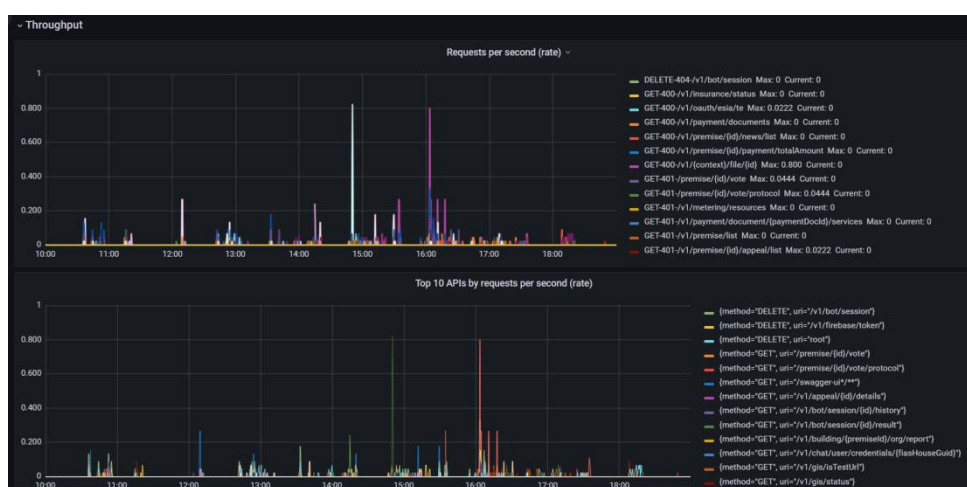


Рисунок 7. Диаграммы, показывающие самые нагруженные эндпоинты API по запросам в секунду

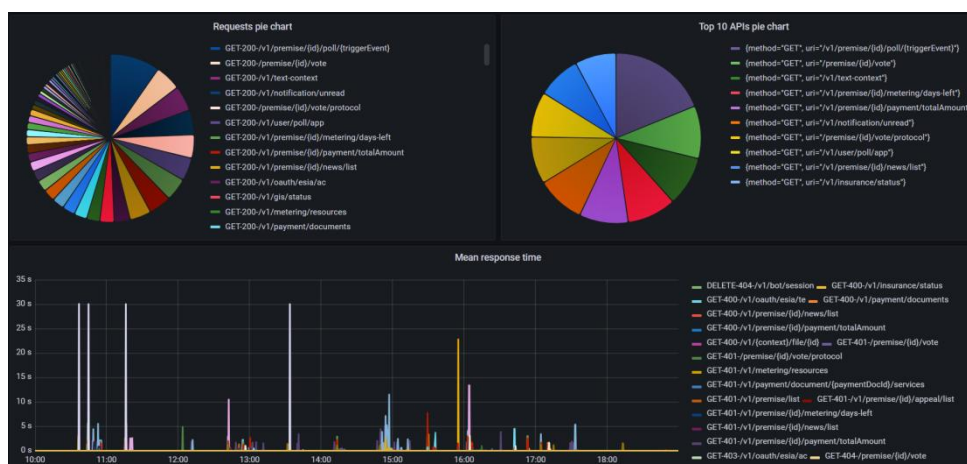


Рисунок 8. Распределение самых нагруженных API и среднее время ответа

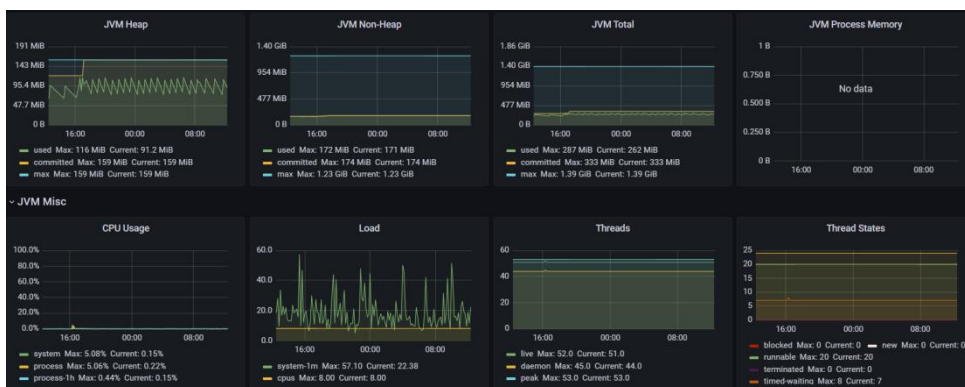


Рисунок 9. Метрики JVM

Также были реализованы уведомления в бот Telegram по определенным событиям, требующим реагирования. Например, если количество запросов превышает определённый порог, то система отправляет уведомление в бот Telegram.

В результате реализации мониторинга удалось контролировать работу системы и предотвращать возможные проблемы. Благодаря использованию Spring Boot Actuator, Prometheus и Grafana удалось создать эффективный механизм мониторинга, который позволяет оперативно реагировать на изменения в работе системы.

4.7 Документирование и тестирование ПО

В качестве средства наглядного представления разработанного API и с целью его документирования, а также удобной отладки в ручном режиме, будем использовать Swagger UI (OpenAPI 3.0).

Swagger – это мощный инструмент для документирования и представления API. Он позволяет автоматизировать процесс генерации интерактивной документации, что значительно упрощает взаимодействие с методами интерфейса. Для использования Swagger в нашем приложении мы добавили зависимость «org.springdoc:springdoc-openapi-ui», которая позволяет автоматически генерировать документацию на основе специальных аннотаций, таких как `@Entity` и `@RestController`. После подключения, документация становится доступна по адресу `/swagger-ui.html`, что позволяет опробовать взаимодействие с методами API прямо в браузере. Это удобное и эффективное решение для разработки и тестирования API.

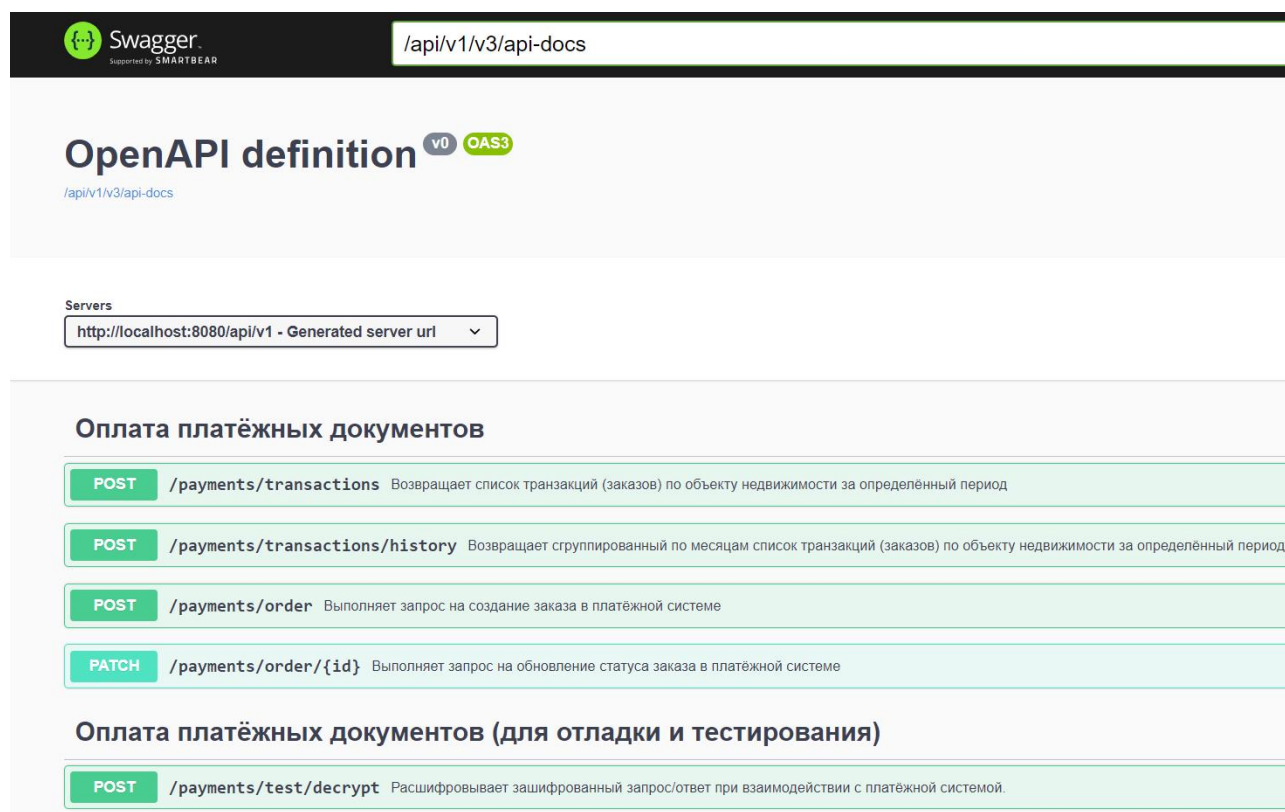


Рисунок 10. Демонстрация работы Swagger на примере разработанного API

В дополнение к Swagger, также подробно были задокументированы методы и классы с помощью Javadoc в исходном коде. Javadoc – это инструмент для генерации документации из

исходного кода Java. Он позволяет создавать описания классов, методов, полей и других элементов кода, а также генерировать HTML-страницы с этой документацией.

Добавление зависимости `org.springdoc:springdoc-openapi-javadoc` позволило автоматически включить Javadoc документацию. Добавление Javadoc документации в исходный код проекта позволяет разработчикам быстрее ориентироваться в коде и понимать его структуру и логику. Кроме того, это упрощает процесс поддержки и развития проекта, так как новым разработчикам будет легче вникнуть в код и начать работать с ним.

Для того чтобы обеспечить высокое качество кода и уверенность в его работоспособности, в проекте было проведено модульное тестирование с помощью Junit и интеграционное тестирование с использованием Wiremock. Эти инструменты позволили выявить множество ошибок и проблем на ранних этапах разработки, что существенно сократило время на отладку и исправление ошибок перед выходом в релиз.

Junit – это фреймворк для модульного тестирования Java-приложений. Он позволил создавать тесты для отдельных модулей приложения и проверять их работу в изоляции от других модулей. Для достижения изоляции в совокупности с Junit также использовался Mockito – фреймворк, позволяющий подменить некоторые объекты *моками* – так называемыми «пустышками».

Wiremock – это инструмент для создания и эмуляции HTTP-серверов и клиентов. В нашем случае он позволил создавать интеграционные тесты, которые проверяли работу приложения в условиях реального окружения посредством эмулирования ответов от сторонних сервисов, что позволяло проверять работу приложения в различных сценариях.

Testcontainers – это библиотека для создания временных контейнеров Docker в тестовом окружении. Она позволила нам создавать отдельные экземпляры БД для каждого теста, что упрощало и автоматизировало процесс тестирования.

Использование Junit, Wiremock и Testcontainers в проекте позволило выявить множество ошибок и проблем на ранних этапах разработки, что существенно сократило время на отладку и исправление ошибок перед выходом в релиз. Эти инструменты обеспечили высокое качество кода и уверенность в его работоспособности, что является важным фактором для успешного развития проекта.

4.8 Описание результатов

Итак, в ходе проделанной работы над поставленной задачей разработки программного модуля в рамках проекта для автоматизации услуг в сфере ЖКХ, разработано приложение (backend сервис), реализующий функциональность оплаты платёжных документов и внедрённый в инфраструктуру проекта.

Представлено детальное описание дизайна моделей данных с учётом функционального дизайна, исходный код сопровождается комментариями для разработчиков, а также наглядной автоматически сгенерированной документацией API с использованием инструментов Swagger. Это позволяет считать приложение легко поддерживаемым и готовым к расширению функционала впоследствии.

Благодаря тому, что параллельно с разработкой серверной части системы другими исполнителями проекта также велась активная разработка модуля, представляющего собой мобильный пользовательский интерфейс, опирающийся на контракт API между сервером и клиентом, были вовремя выполнены обязательства перед заказчиком.

Таким образом, одну из целей данной выпускной квалификационной работы можно считать достигнутой.

5 Разработка программного модуля для функциональности поверки приборов учёта

В данном разделе мы рассмотрим процесс разработки серверной части программного модуля, который обеспечивает функциональность поверки индивидуальных приборов учёта (см. раздел 1.4). Результатом этой работы будет готовый к интеграции модуль, который можно будет использовать в составе системы разрабатываемого проекта.

Однако следует отметить, что разработка клиентской части, то есть мобильного приложения, не является частью данной работы. Эта задача была выполнена другими разработчиками на основе интерфейса, созданного в результате работы над серверной частью, описываемой в данном разделе.

Как было отмечено ранее, необходимо обратить внимание, что из-за NDA (обязательства неразглашения конфиденциальной информации) невозможно предоставить полную информацию о технической реализации и модели данных. Эта информация будет представлена в упрощённом и изменённом виде, что позволит дать общее представление, при этом не нарушив обязательство о конфиденциальности. Это означает, что может быть упущено некоторое количество деталей, но общая структура и характеристики модели будут представлены и объяснены. Цель такой модификации данных – сохранить конфиденциальность и защитить права заключивших NDA сторон.

5.1 Постановка задачи

В рамках производственной необходимости проекта требуется разработать программный модуль (backend), который будет обеспечивать функциональность подачи пользователем заявки на поверку прибора учёта (счётчика) посредством интеграции с внешним платёжным сервисом.

Разрабатываемый программный модуль должен предоставить API для мобильного приложения, позволяющий запросить стоимость процедуры поверки, зависящей от региона пользователя, и в случае согласия направить заявку на проведение процедуры поверки специалистом внешней компании.

Для обеспечения синхронизации статусов заявок с внешней системой поверки необходимо разработать механизм обновления статусов заявок по внешнему запросу.

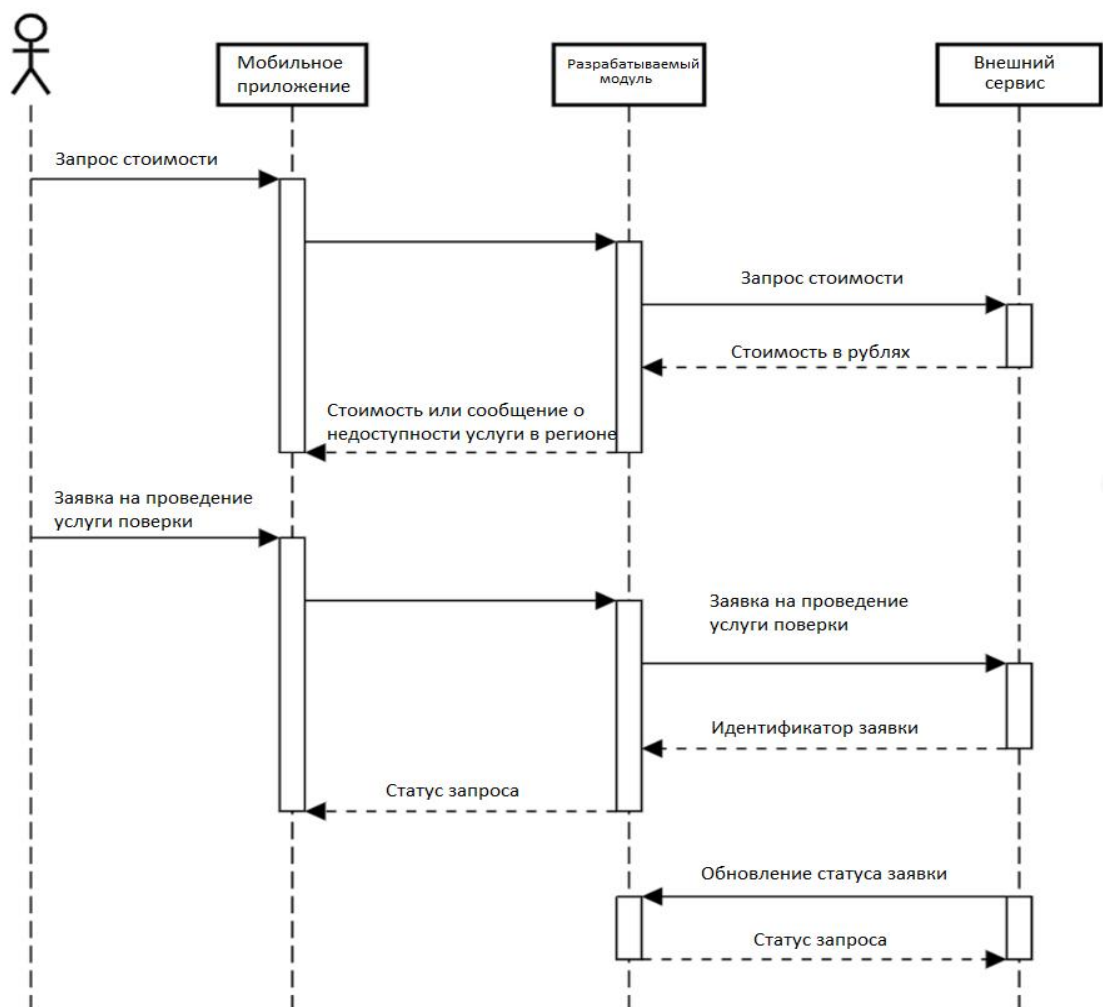


Рисунок 11. Диаграмма последовательности запросов процедуры поверки прибора учёта

Взаимодействие с внешней системой должно быть защищено шифрованием для обеспечения безопасности передачи данных.

Для обеспечения надёжности и отслеживаемости процесса подачи заявок необходимо осуществлять подробное логирование. Кроме того, информация о заявках должна храниться в базе данных, что требует разработки соответствующей модели хранения данных.

Таким образом, задачей данного раздела является разработка серверной части программного модуля для обеспечения функциональности подачи заявок на проведение процедуры поверки приборов учёта через внешний платёжный сервис. В рамках проекта необходимо разработать API для мобильного приложения, механизм обновления заявок, защищённое взаимодействие с внешней системой, подробное логирование и модель хранения данных.

5.2 Дизайн моделей данных

Как известно, разработка любого компонента программного обеспечения является сложным процессом, в котором необходимо учитывать множество факторов. Одним из таких факторов является корректное фиксирование дизайна моделей данных, которое играет важную роль в создании готового к интеграции модуля.

Важность корректного фиксирования дизайна моделей данных заключается в том, что разрабатываемый компонент должен быть готов к интеграции в большую систему, разрабатываемую командой других программистов. При этом от него ожидается продуманный дизайн интерфейсов API и моделей данных, который гарантирует отсутствие нарушения «контрактов» с другими модулями, завязавшимися на функционал текущего.

В связи с этим, при разработке проекта было принято решение зафиксировать следующий дизайн моделей данных, задействованных в функционале разрабатываемого сервиса. При этом необходимо понимать, что обсуждение дизайна моделей данных представляет собой динамический процесс, который требует проведения нескольких итераций ввиду постепенного выявления требований. Поэтому в данном разделе представляется уместным документирование лишь заключительной версии дизайна, без протоколирования промежуточных наработок.

5.2.1 Дизайн моделей данных, связанных с заявками на поверку

В данном подразделе приводится дизайн типа данных, который моделирует сущность, представляющую собой заявку на поверку. Для удобства данные структурированы в табличном виде.

Таблица 8

Тип данных AppealForVerification		
<i>Данный тип данных представляет собой сущность, описывающую заявку на проведение поверки.</i>		
Имя поля	Тип поля	Описание
id	Строковый	Уникальный идентификатор созданной заявки во внутренней системе (автоматически генерируемый первичный ключ записи).
status	ExtAppealStatus	Статус заявки на поверку.
created	Дата и время	Дата и время, когда заявка была создана.
updated	Дата и время	Дата и время, когда заявка была последний раз изменена.
device_id	Строковый	Идентификатор (во внутренней системе) прибора учёта, к которому относится заявка на поверку.
device_number	Строковый	Номер прибора учёта, к которому относится заявка на поверку.
user_id	Строковый	Идентификатор (во внутренней системе) пользователя, создавшего заявку на поверку.
property_id	Строковый	Идентификатор (во внутренней системе) объекта недвижимости, к которому относится заявка на поверку.
external_id	Строковый	Идентификатор созданной заявки во внешней системе.
external_number	Строковый	Номер заявки во внешней системе.
calibration_date	Дата	Дата, когда поверка должна быть проведена по плану.
verification_date	Дата	Дата, на которую фактически назначена процедура прохождения поверки.

Таблица 9

Тип данных ExtAppealStatus		
<i>Перечислимый тип данных, представляющий собой статус заявки.</i>		
Значение	Описание	Возможные переходы по статусной модели
CREATED	Заявка создана, но ещё не зарегистрирована во внешней системе проверок.	REGISTERED
REGISTERED	Заявка успешно зарегистрирована во внешней системе проверок, но к работам ещё не приступили и дата процедуры ещё не назначена.	IN_PROGRESS
IN_PROGRESS	Заявка в работе (назначена дата процедуры проверки).	CANCELLED, DONE
CANCELLED	Заявка отменена.	—
DONE	Заявка исполнена.	—

5.3 Функциональный дизайн API

Рассматриваемая глава раскрывает результаты анализа функциональных требований, предъявляемых к серверной части создаваемого модуля. Важность документирования программного обеспечения в больших системах не может быть переоценена, поскольку время поддержки может значительно превышать время, затрачиваемое на разработку. Хорошо сформулированный функциональный дизайн модуля позволяет определить его «обязанности» и гарантировать, что он не повлияет на другие модули системы, что в целом повышает стабильность и устойчивость системы, а также облегчает последующую корректировку функционала.

В отношении создаваемого модуля сервиса заявки на поверку наличие функционального дизайна API позволяет установить «контракт» между серверной и клиентской частями приложения. Это особенно актуально, поскольку разработка клиентской части (мобильного приложения) выполняется другими участниками проекта и выходит за рамки данной работы.

Таблица 10.

Название операции	HTTP метод	Путь	Входные данные	Выходные данные
Выполняет запрос от мобильного приложения на получение информации о стоимости (и возможности) услуги поверки для заданного дома.	GET	/appeal/verification/price	В параметре запроса передаётся идентификатор объекта недвижимости, к которому относится прибор учёта, по которому необходимо провести процедуру поверки.	В теле ответа передаётся стоимость запрашиваемой услуги, либо сообщение об ошибке, что по данному адресу оказание услуги недоступно (зависит от региона).
Запрос на смену статуса заявки. Не для мобильного приложения, а для внешней системы.	PATCH	/appeal/verification/{id}/status	В параметрах пути запроса передаётся уникальный идентификатор заявки, а в теле запроса – новый статус заявки, а также новая дата проведения поверки.	—
Запрос на создание заявки на поверку.	POST	/appeal/verification	В теле запроса передаётся структура данных, содержащая в себе все необходимые атрибуты (включая идентификаторы подлежащего поверке прибора	Структура данных, представляющая собой идентификатор и номер созданной заявки (либо сообщение об ошибке).

			учёта и объекта недвижимости, а также контактные данные пользователя) для выполнения запроса во внешнюю систему и сохранения сущности заявки в базу данных.	
Возвращает детальную информацию о заявке на поверку.	GET	/appeal/verification/{id}	В параметре пути запроса передаётся идентификатор заявки.	Структура данных, представляющая собой детальную информацию о заявке на поверку (преобразованную из соответствующей сущности в базе данных).

Для описания API выбран архитектурный стиль взаимодействия компонентов REST. Наиболее наглядным методом описания API является таблица, содержащая сводку по доступным операциям API с указанием HTTP протокола, по которому осуществляется запрос, пути относительно корневого адреса сервера, входных (параметры пути, параметры запроса, тело запроса) и выходных данных.

Одним из основных требований к создаваемому модулю является обеспечение безопасности. Для этого требуется, чтобы доступ к каждой из операций REST API был разрешён только для авторизованных пользователей. Пользователь предварительно выполняет запрос аутентификации, чтобы авторизоваться в системе. Авторизованным запросом считается тот, в заголовке (header) которого верно переданы закодированные аутентификационные данные.

5.4 Программная реализация

В данном разделе рассматривается непосредственная реализация компонента, функционал и дизайн которого детально описан в разделах 5.2 (модели данных) и 5.3 (функциональные требования).

В качестве языка программирования разрабатываемого приложения используется Java версии 17 (Open JDK). Фреймворк – Spring Boot версии 2.7.5.

5.4.1 Высокоуровневая архитектура сервиса

В разделе 4.4.1 приведено подробное объяснение шаблона проектирования, предусматривающего разделение ответственности между различными компонентами приложения (разделение на уровень доступа к базе данных, работу с бизнес-логикой и предоставление доступа к своей функциональности для внешних сервисов).

В этом плане архитектура выглядит подобным образом, с той разницей, что для обработки внешнего запроса на обновление статуса был создан специальный gateway для внешних интеграций, чтобы можно было удобно разграничивать доступ на сетевом уровне. Таким образом, запрос закрыт для мобильного приложения, но открыт для конкретной системы.

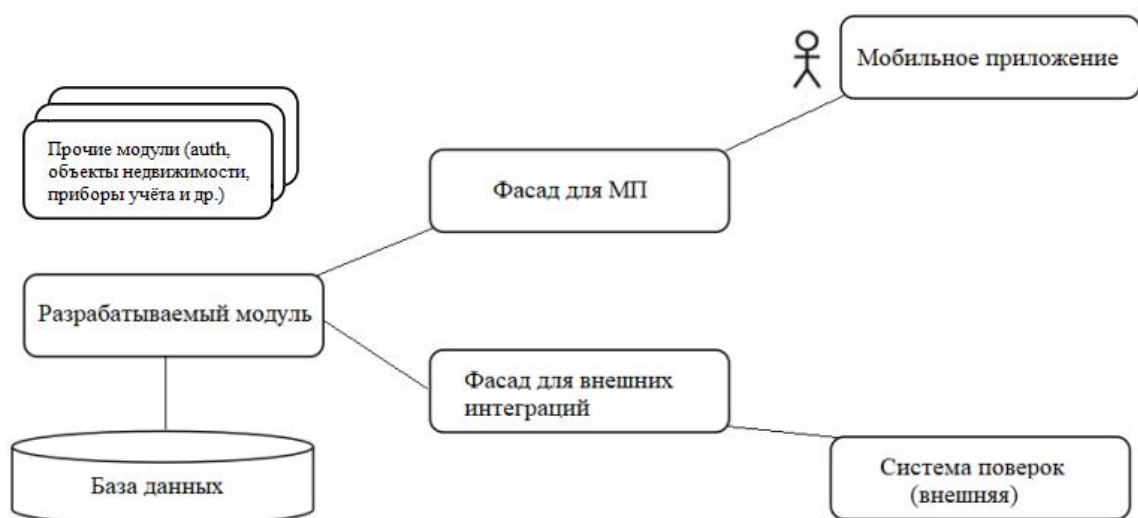


Рисунок 12. Архитектура программных модулей

Итак, с учётом введённых определений, высокоуровневую архитектуру подмодулей разрабатываемого сервиса можно упрощенно изобразить следующим образом:

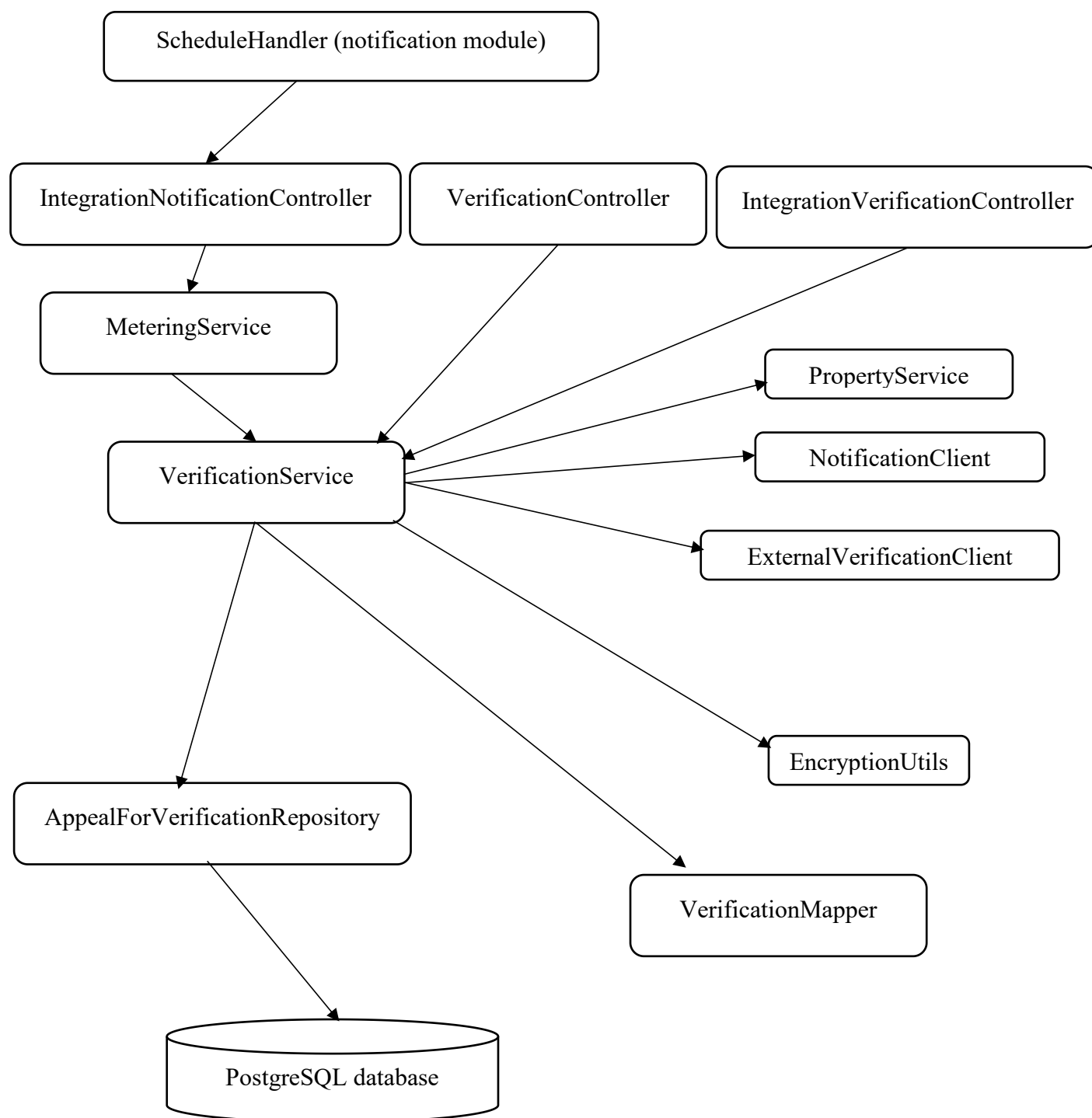


Рисунок 13. Высокоуровневая архитектура разрабатываемого сервиса заявок

5.4.2 Детали технической реализации требуемой функциональности

Проведя первоначальный анализ высокоуровневой структуры компонентов нашего приложения, рассмотрим некоторые важные детали программной реализации ключевых функций, в частности REST API для работы с заявками на поверку.

Методы контроллера `VerificationController` являются точками входа в программу из мобильного приложения и полностью соответствуют функциональному дизайну, описанному в разделе 5.3. Внутри тела этих методов происходит обращение к соответствующим методам в классе `VerificationService`.

Среди функциональных задач, реализуемых в разрабатываемом модуле, можно выделить следующие:

- При получении запроса от мобильного приложения на создание заявки, перед отправкой заявки во внешнюю систему, решается задача по распознаванию, не является ли запрос пользователя дубликатом уже инициированной заявки. Дубликаты могут возникнуть, во-первых, из-за того, что пользователь пытается повторно подать заявку по конкретному прибору учёта (человеческий фактор), а во-вторых, в результате ранее неудачно отправленного запроса и его повторной попытки (технический фактор). В первом пользователь получает информацию об уже созданной заявке, а во втором случае «незаконченная» заявка извлекается из базы данных, и уже она продолжает обрабатываться перед отправкой во внешнюю систему сервиса проверок.
- Формирование сущности `AppealForVerification` и сохранение в БД в промежуточном статусе (принято от мобильного приложения, но не зарегистрировано во внешней системе).
- Формирование запроса во внешнюю систему поверки (маппинг). При формировании запроса на создание заявки на поверку во внешнюю систему, происходит «обогащение» полученных из мобильного приложения атрибутов запросов дополнительными данными из отдельного модуля, отвечающего за объекты недвижимости, происходит получение идентификатора дома, в котором находится подлежащий поверке прибор учёта. Аналогично происходит в методе запроса стоимости услуги (там это необходимо для выявления возможности оказания услуги по данному адресу).
- Запрос во внешнюю систему и обработка результатов: в случае успеха статус заявки в БД меняется на успешно отправленную (`REGISTERED`) и мобильное приложение получает номер созданной заявки, в противном случае статус

заявки в БД остаётся просто созданным (CREATED), и мобильное приложение получает ответ с ошибкой. Как отмечалось ранее, при повторном запросе обработка возобновится.

- Приведение статусов заявок в актуальное состояние в соответствии с данными внешнего сервиса проверок.
- Поиск по расписанию приборов учёта, которые в ближайшее время подлежат проверке, и рассылка PUSH уведомлений на устройства пользователей.
- Решение проблемы «отставания» статуса поверки прибора учёта по данным государственной информационной системы и статуса по данным коммерческой организации, занимающейся процедурой поверки. Иногда такое расхождение может занимать порядка двух месяцев. В связи с этим возникла необходимость разработки и вычисления специальных внутренних статусов приборов учёта, на основании которых можно было отображать пользователю информационное сообщение о том, что поверка проведена, но информация об этом ещё не обновилась. Это нужно для того, чтобы пользователь мог продолжать спокойно пользоваться функциональностью передачи показаний.

5.5 Описание результатов

Итак, в ходе проделанной работы над поставленной задачей разработки программного модуля в рамках проекта для автоматизации услуг в сфере ЖКХ, разработан программный модуль, реализующий функциональность заявок на поверку приборов учёта и внедрённый в инфраструктуру проекта.

Представлено детальное описание дизайна моделей данных с учётом функционального дизайна, исходный код сопровождается комментариями для разработчиков, а также наглядной автоматически сгенерированной документацией API с использованием инструментов Swagger. Это позволяет считать приложение легко поддерживаемым и готовым к расширению функционала впоследствии.

Благодаря тому, что параллельно с разработкой серверной части системы другими исполнителями проекта также велась активная разработка модуля, представляющего собой мобильный пользовательский интерфейс, опирающийся на контракт API между сервером и клиентом, были вовремя выполнены обязательства перед заказчиком.

Таким образом, ещё одну из целей данной выпускной квалификационной работы можно считать достигнутой.

ЗАКЛЮЧЕНИЕ

В рамках работы над проектом по автоматизации услуг жилищно-коммунального хозяйства было решено множество прикладных задач, которые требовали применения различных навыков программной инженерии. Одной из таких задач было создание серверной части программного модуля, который позволил реализовать функциональность оплаты платёжных документов.

Для того чтобы успешно решить эту задачу, был проведён анализ предметной области, в ходе которого были выявлены требования заказчика. После этого была разработана архитектура программных компонентов и созданы модели данных, которые позволили реализовать необходимую функциональность.

В результате проделанной работы удалось удовлетворить поставленные требования и достичь цели данной выпускной квалификационной работы. Полученный результат устроил заказчика, что свидетельствует о качественном выполнении проекта. Разработанный модуль был успешно внедрён в проект и выведен в промышленную эксплуатацию на широкую аудиторию.

Кроме того, в рамках работы над проектом решались такие задачи, как реализация функционала передачи показаний, создание заявок на поверку приборов учёта (счётчиков), отправка уведомлений пользователям и множество других.

Выполнение данной работы позволило применить на практике теоретические знания, освоить современные подходы к разработке с применением облачной микросервисной архитектуры, а также вникнуть в возникающие при этом проблемы и научиться их решать. Этот опыт является ценным в становлении как профессионала и позволит в будущем успешно решать сложные задачи в области программной инженерии.

СПИСОК ЛИТЕРАТУРЫ

1. Белозеров С. А. Модернизация сферы ЖКХ как фактор повышения уровня жизни населения России / С. А. Белозеров, А. Э. Ващук // Уровень жизни населения регионов России. - 2017. - № 12. - С. 47-57.
2. Теоретические аспекты исследования проблем и перспектив внедрения информационных технологий в структуры ЖКХ / С. А. Кирсанов, С. В. Краснов, А. С. Краснов // Экономика и упр. - 2018. - № 12. - С. 15-22.
3. Ветрова, Е. А. Управление жилищно-коммунальным хозяйством : учебно-методическое пособие / Е. А. Ветрова, Е. Е. Кабанова. – Москва ; Берлин : Директ-Медиа, 2021. – 224 с.
4. Олег Нечай. Все говорят о микросервисной архитектуре приложений. Чем она хороша и как на нее перейти? // «Тэдвайзер». – Режим доступа: https://www.tadviser.ru/index.php/Статья:Микросервисная_архитектура:_характерные_особенности,_достоинства_и_недостатки
5. Дарья Барышева. Простым языком о микросервисной архитектуре для начинающих. // Журнал Mail.ru Cloud Solutions. – Режим доступа: <https://mcs.mail.ru/blog/prostym-jazykom-o-mikroservisnoj-arhitekture>
6. Максим Рокатанский. Лучшая архитектура для MVP: монолит, SOA, микросервисы или бессерверная?.. Часть 1 // «Habr». – Режим доступа: <https://habr.com/ru/company/otus/blog/476024/>
7. Craig Walls. Spring in Action, 5th edition. – USA: Manning Publications Co, 2019. – 499 с.
8. Spring Boot // Материал из Национальной библиотеки им. Н. Э. Баумана. – Режим доступа: https://ru.bmstu.wiki/Spring_Boot
9. Lisa Smith. What PostgreSQL has over other open source SQL databases: Part I. // Официальный сайт компании Compose. – Режим доступа: <https://www.compose.com/articles/what-postgresql-has-over-other-open-source-sql-databases/>
10. Сильбершац, А., Корт, Х. Ф., & Сударшан, С. (2019). Концепции систем управления базами данных. – М.: Вильямс, 1376 с.
11. Грэй, Дж., & Ройтер, А. (1993). Обработка транзакций: концепции и методы. – Morgan Kaufmann, 1128с.
12. Документация Spring Scheduler [Электронный ресурс]. – Режим доступа: <https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/scheduling/annotation/Scheduled.html>

13. Документация Quartz Scheduler [Электронный ресурс]. – Режим доступа:
<https://www.quartz-scheduler.org/documentation/>
14. Документация Spring Framework [Электронный ресурс]. – Режим доступа:
<https://docs.spring.io/spring-framework/docs/current/reference/html/index.html>
15. Ferguson, N. Cryptography Engineering: Design Principles and Practical Applications / N. Ferguson, B. Schneier, T. Kohno. – США: Wiley Publishing, 2010. – 384 с.
16. Daemen, J. The Design of Rijndael: AES - The Advanced Encryption Standard / J. Daemen, V. Rijmen. – США: Springer-Verlag New York, Inc., 2002. – 432 с.